구조 설계 및 평가

조 용 진

(drajin.cho@bosornd.com)

BY NC ND

강사 소개

성명	조용진(CHO YONG JIN) – 소프트웨어 플랫폼 전문가/아키텍트
연락처	drajin.cho@bosornd.com (+82-10-3754-2879)
학력	1994년 2월 한국과학기술대학 전산학과 졸업 1996년 2월 한국과학기술원 전산학과 석사 졸업 1998년 2월 한국과학기술원 전산학과 박사 수료
경력	2003년 ~ 2016년 2월, (주)삼성전자 근무 스마트 홈 & 상황인지 서비스 프레임워크 개발(2003년~2004년) 디지털 TV 소프트웨어 플랫폼 개발 및 상품화(2005년~2009년) 인도 델리 연구소 주재(2010년~2014년) 사물인터넷 기기를 위한 Tizen 플랫폼 개발(2015년) 2016년 11월 22일 주식회사 보이는소프트웨어연구소 창업 삼성전자 소프트웨어 구조설계자 양성 및 인증(2016년~현재)

목차 – 구조 설계 및 평가(핵심 개념)

repository: https://github.com/bosornd/arch-design-examples

1) 구조 명세 – 구조도

2) 구조 평가 – 품질 요구사항, 구조적 의사결정

3) 재설계 – ADR (Architecture Decision Records)

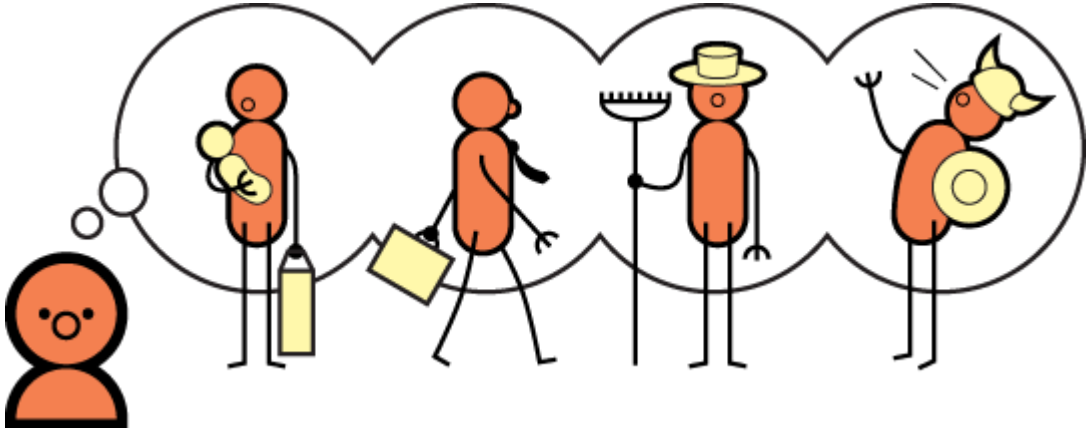
4) 구조 설계와 문서화

5) arch-with-ai (https://github.com/bosornd/arch-with-ai)

Role of Architect

BY NC ND

Role of Architect?



Whoever knows SW cannot say it’s easy.

Architect makes SW easier.

Writing vs. Programming

Learning English	Learning programming language
Writing diary	Writing sample program
Writing a short story	Developing an Android app
Writing a novel	Developing an JavaScript engine
Writing an epic	Developing Web browser

Writing vs. Programming

Learning English

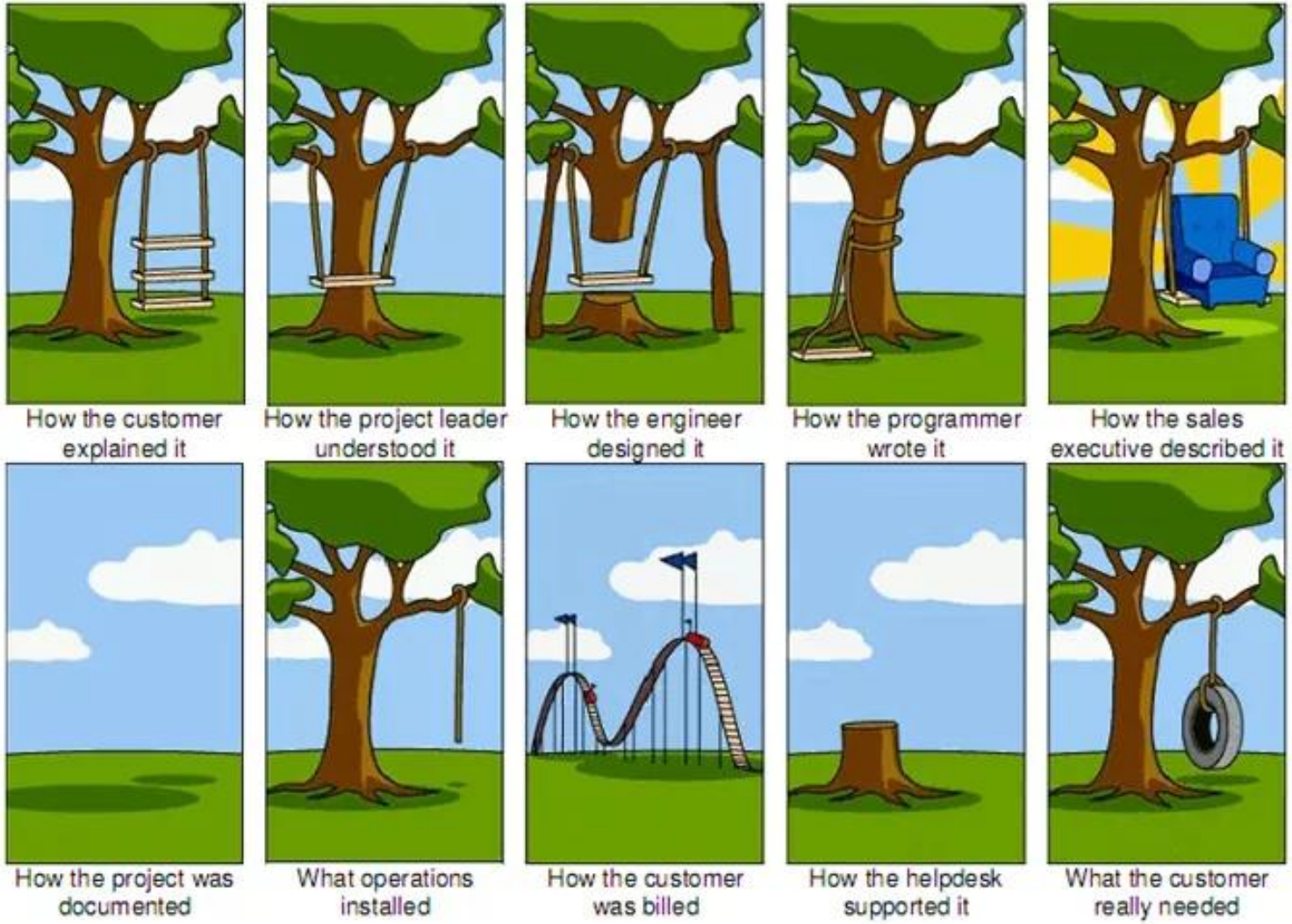
Learning programming language

Writing a novel

more difficult

Developing Software

Why SW is Difficult?



Why SW is Difficult?

Because of Requirements

- Ambiguous
- Incomplete / missing
- Changing

How about requirements of the building?

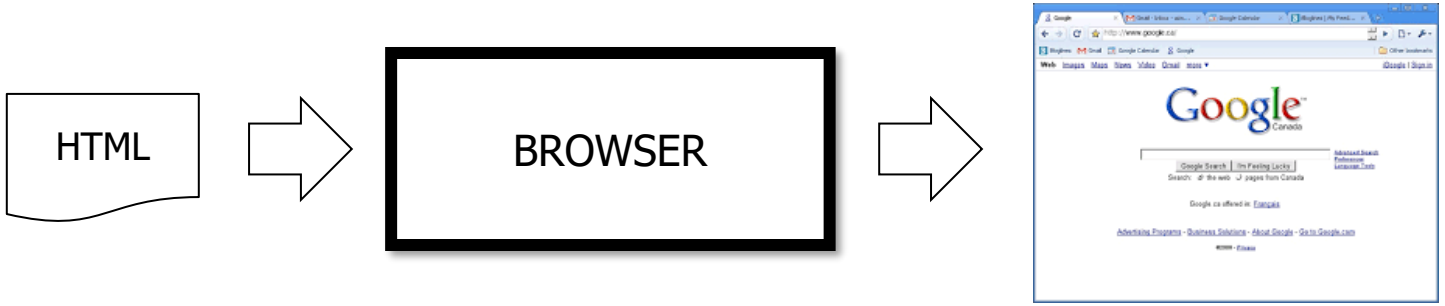
- Nothing ambiguous?
- Everything complete?
- No changes in the construction?

Why SW is Difficult?

If requirements are clear, …

Why SW is Difficult?

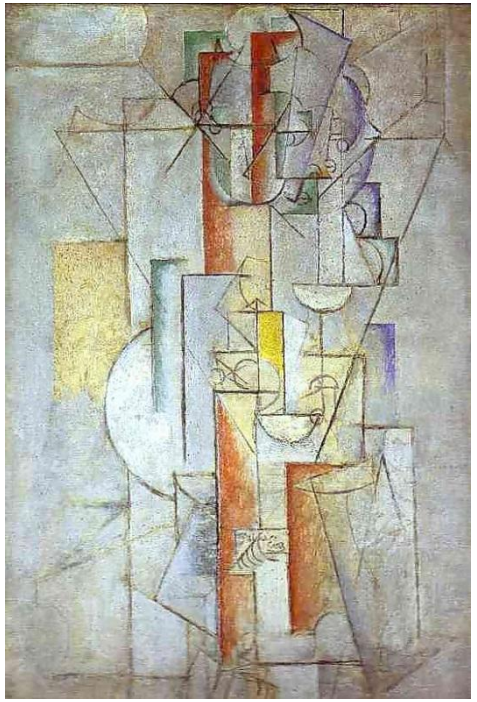
If requirements are clear, …



Something rather than requirements makes SW difficult.

What makes SW more Difficult?

Picasso Paintings

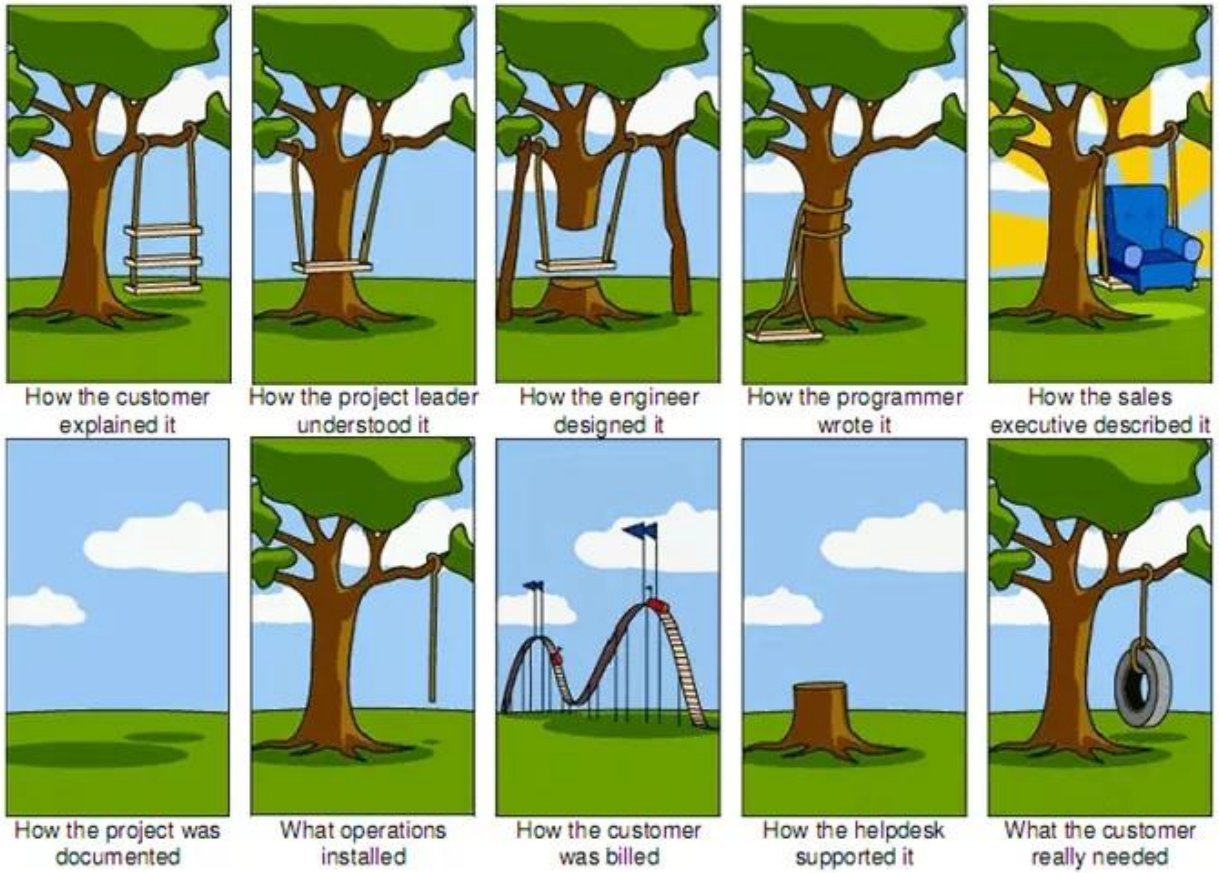


Nude, I love Eva

Woman Playing the Mandolin

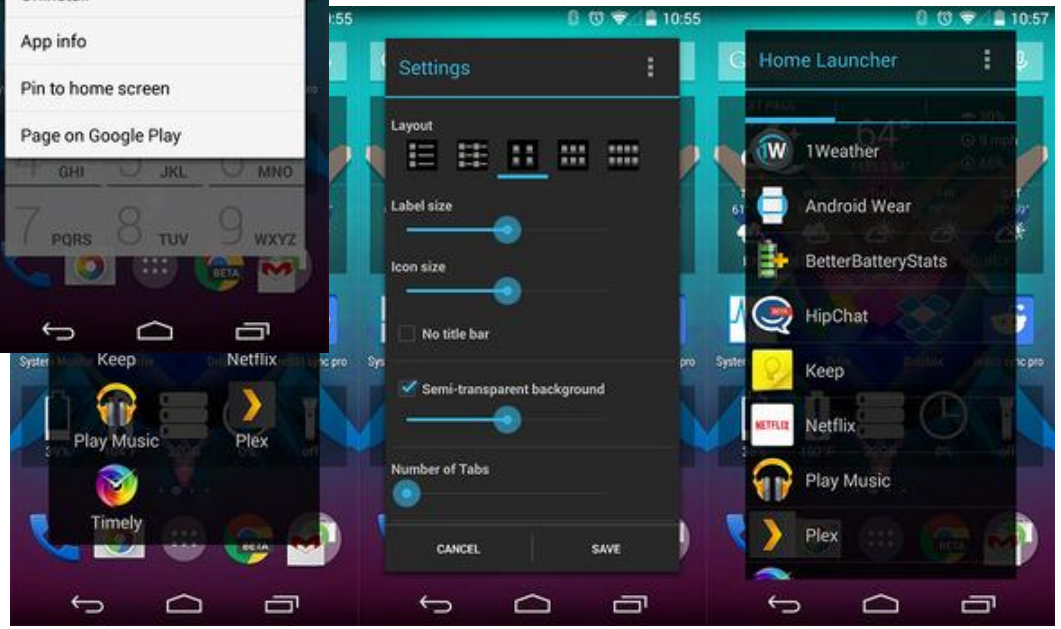
What makes SW more Difficult?

SW is [ ] in nature.



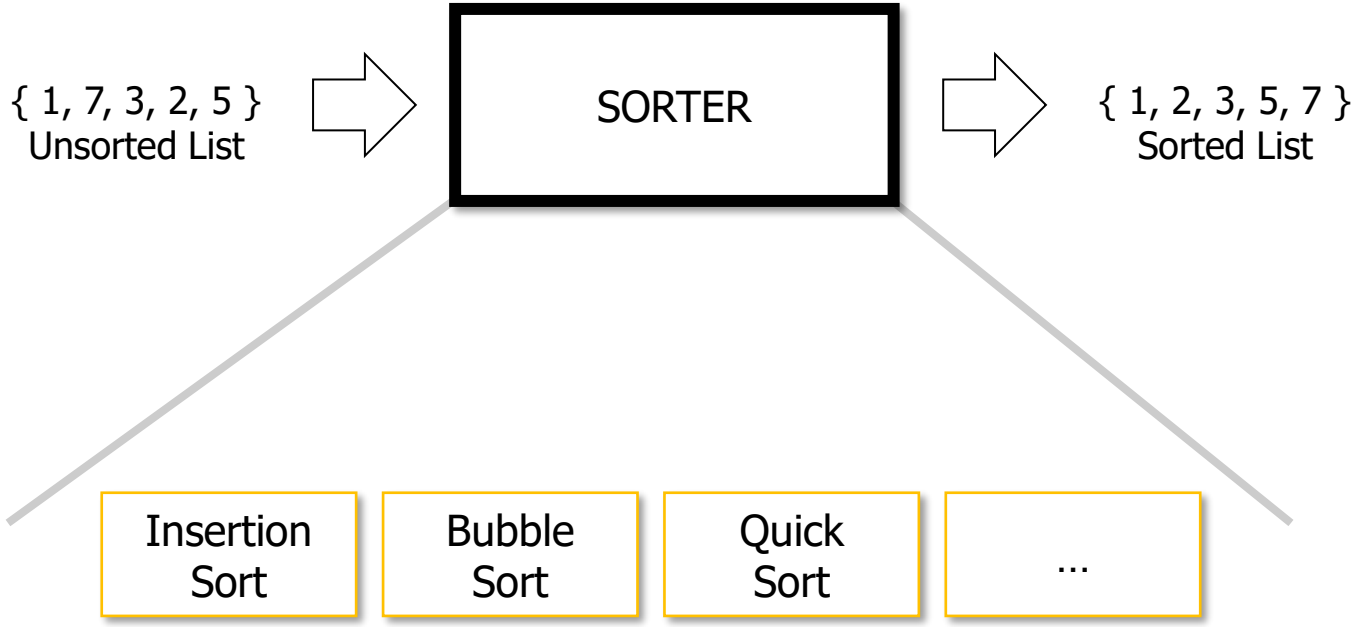
What makes SW more Difficult?

SW is invisible in nature.



What makes SW more Difficult?

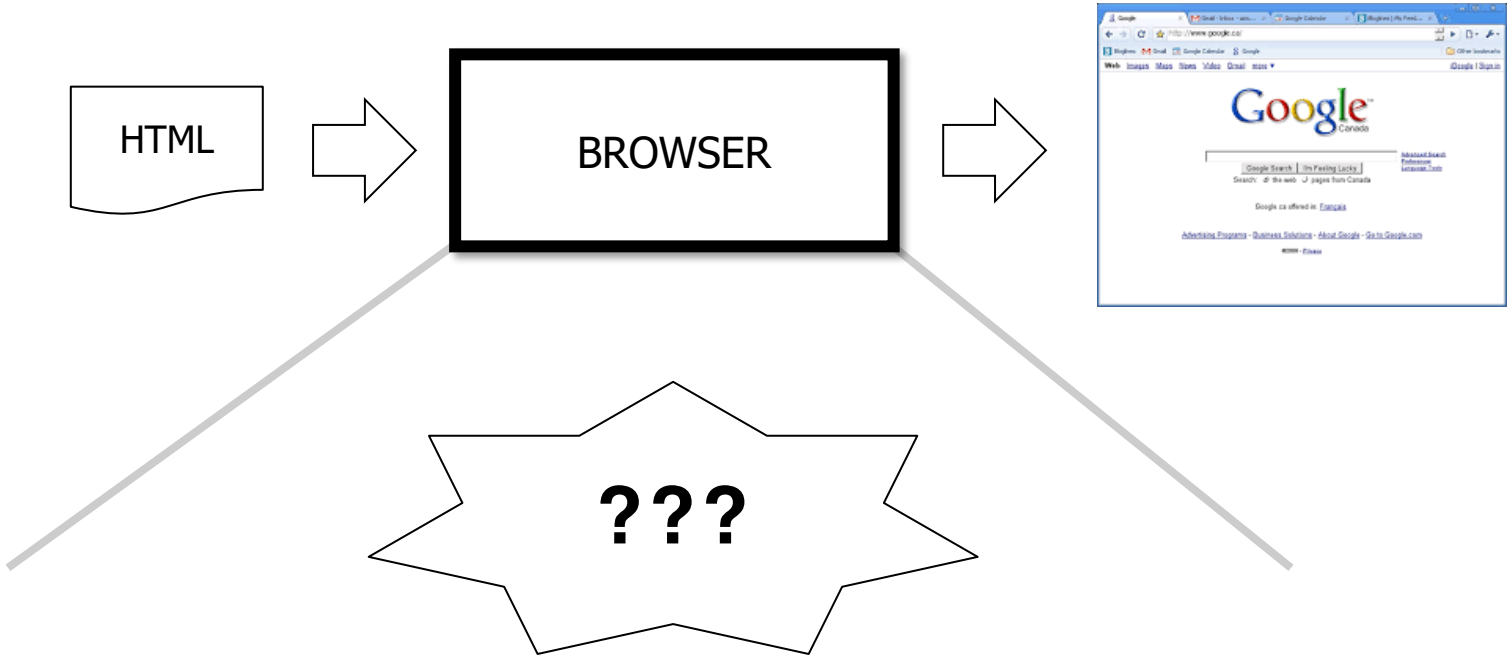
SW is invisible in nature.



What makes SW more Difficult?

SW is invisible in nature.

➔ Hard to imagine what it's composed of and how they work



What makes SW more Difficult?

SW is invisible in nature.

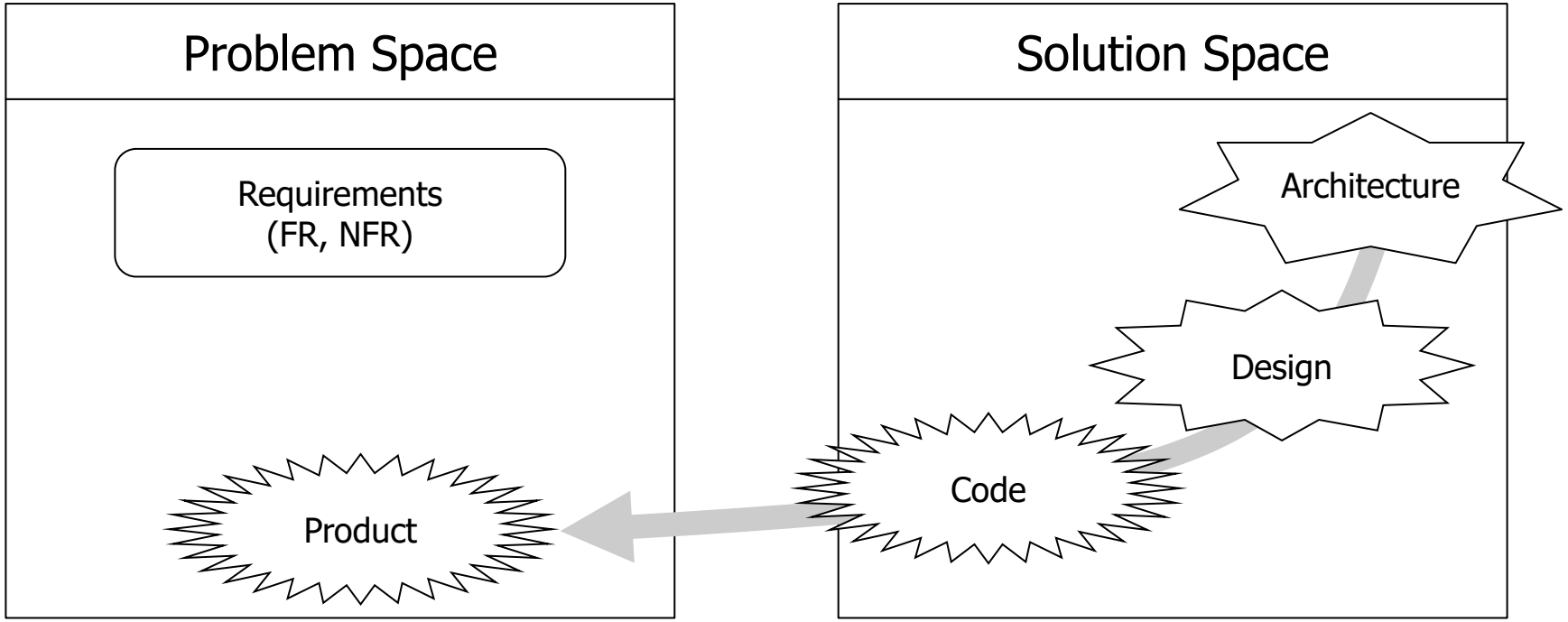
➔ Hard to imagine what it's composed of and how they work

Problem Space	Solution Space
HTML X	Y=F(X)
Y	??? F

What makes SW more Difficult?

SW is invisible in nature.

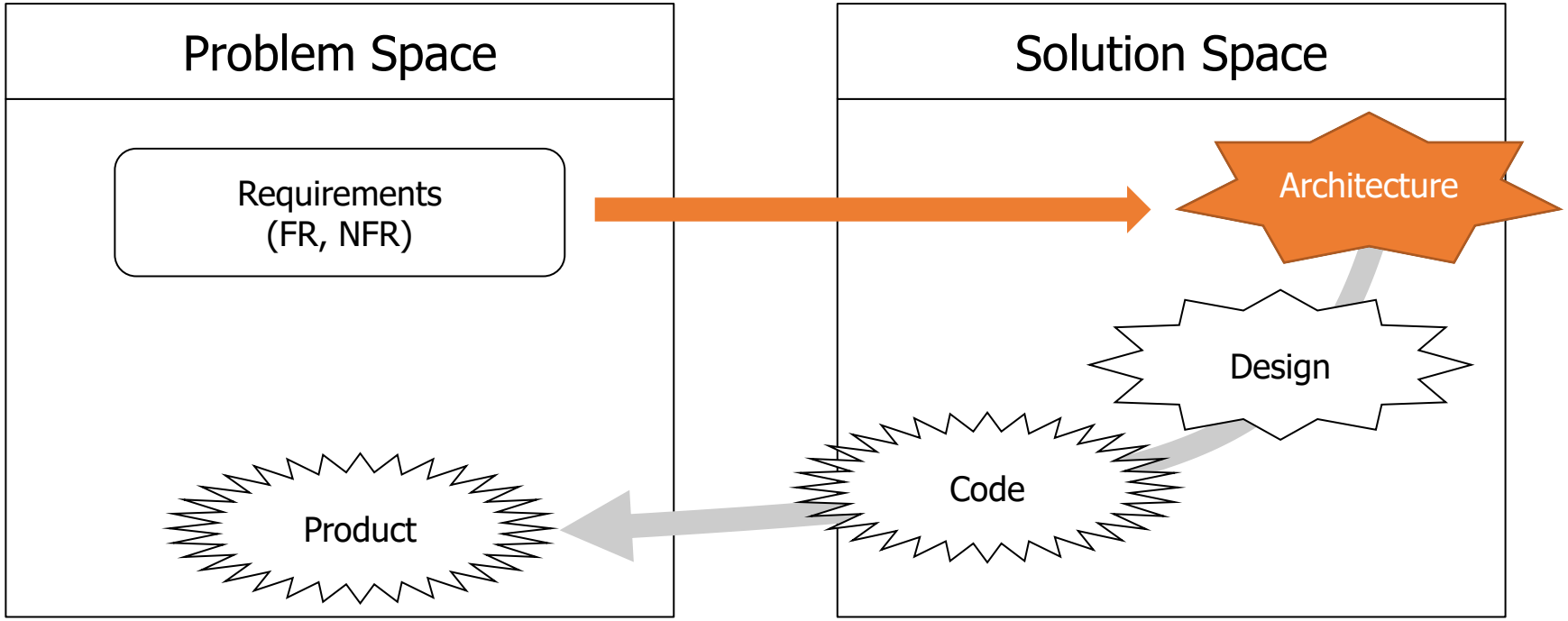
➔ Hard to imagine what it's composed of and how they work



Role of Architect

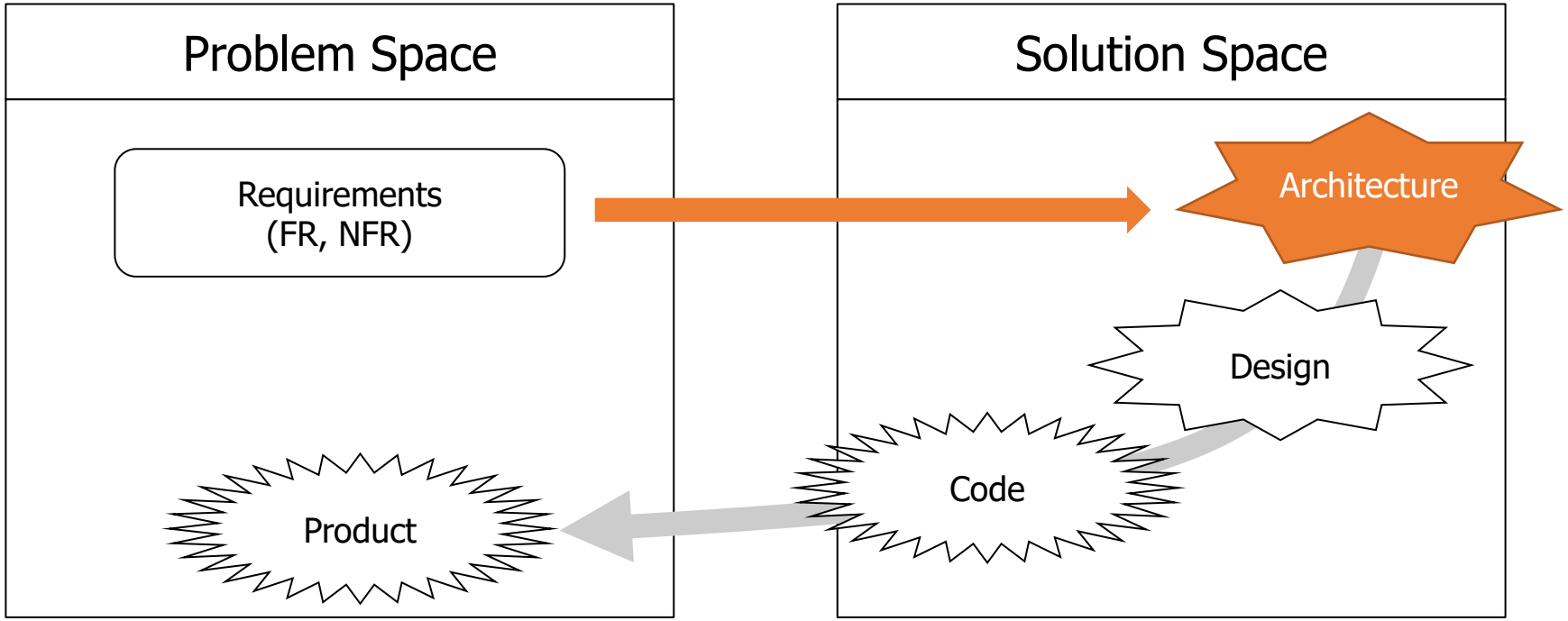
Architect shows what it's composed of and how they work (architecture)

➔ Make SW imaginable and easier to develop



Way of Architect

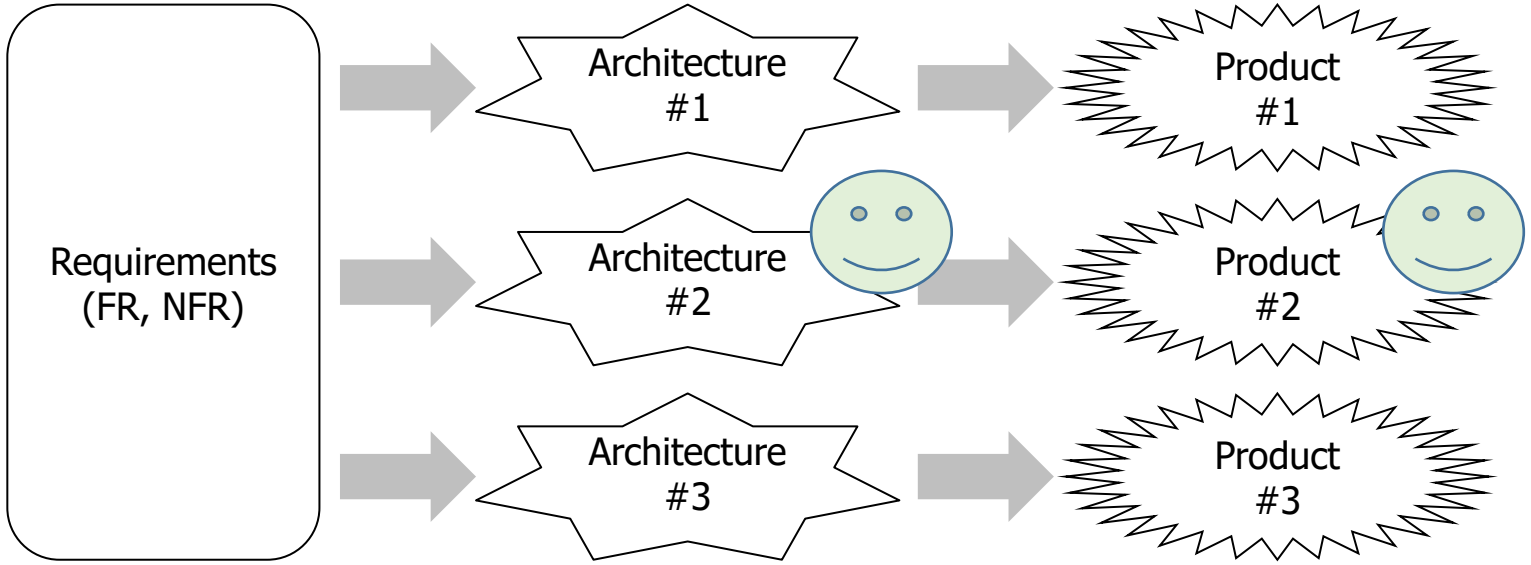
Architect tries to [ ] what it's composed of and how they work and find the best one for the project



Way of Architect

To find the best architecture,

architect evaluates architecture candidates by



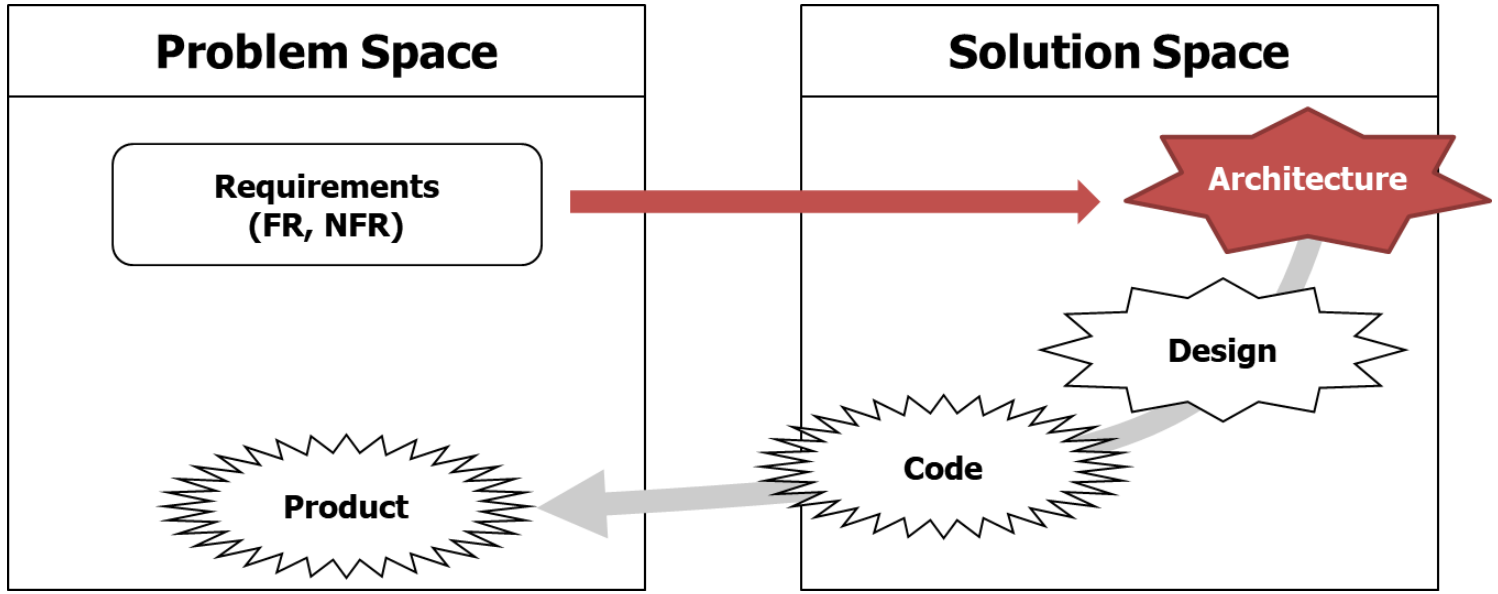
Why Architecture more concerned?

Architecture change is more expensive than design or code change.

How about requirement changes?

Cost depends on implementation change.

- architecture change
- design change
- code change



Summary

Architect shows

what the required SW system is composed of and how they work

🡐 Architecture

evaluates architecture candidates and finds the best one for the project [business]

makes easier to construct and maintain the required SW system [development]

SW is More Difficult, Why?

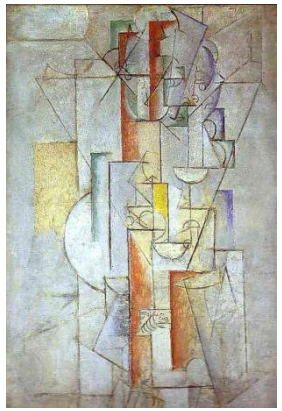
Learning English

Writing a novel

Learning programming language

Developing software

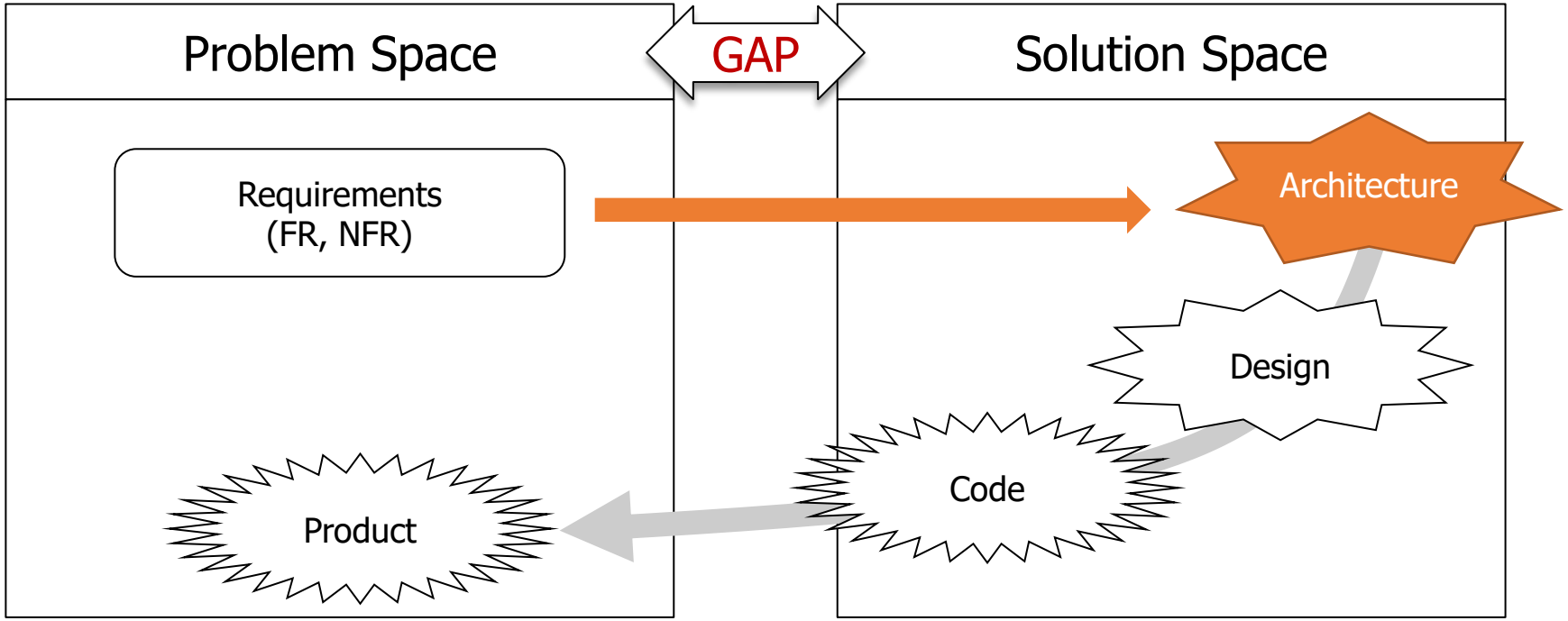
more difficult



More Difficult?

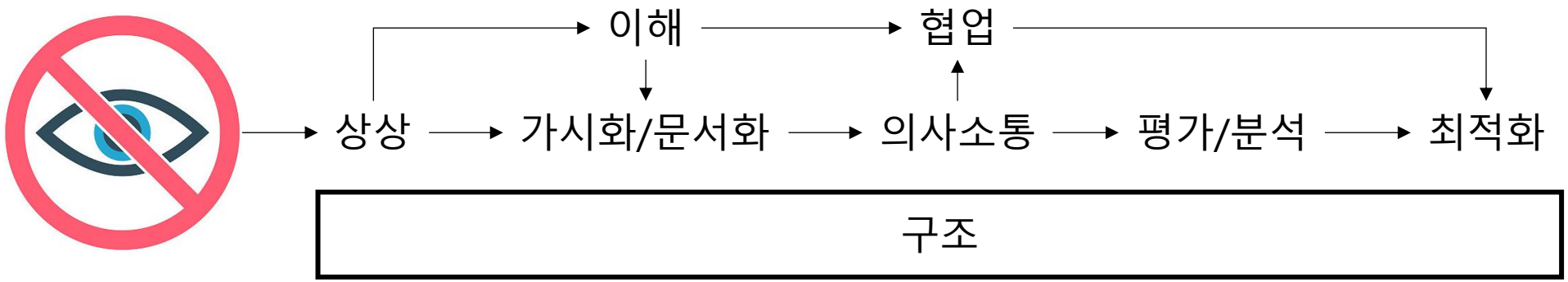
Difficulty ∝ GAP between problem space and solution space

Expert(Architect) is familiar with solution space



아키텍트의 역할

- 구조도를 그린다.



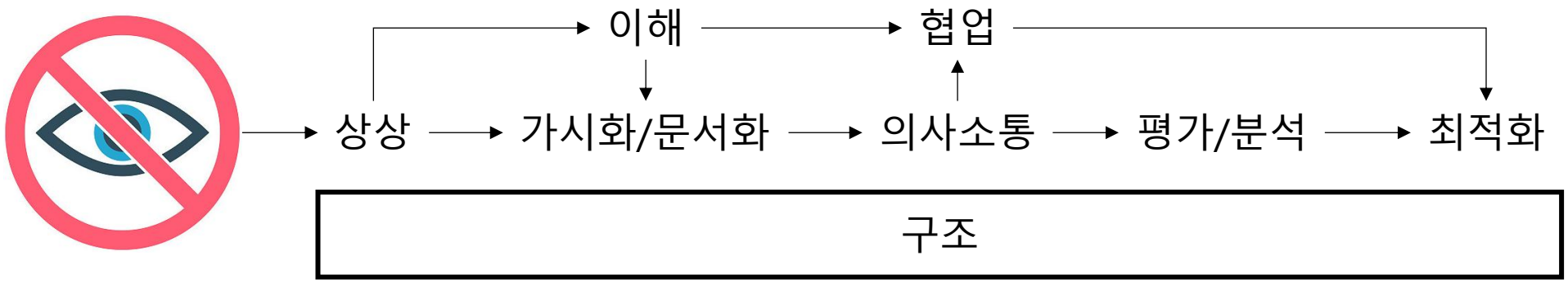
- 최적 설계/개발을 이끈다.

구조도

BY NC ND

아키텍트의 역할

- 구조도를 그린다.



- 최적 설계/개발을 이끈다.

Software Architecture

The software architecture of a system is

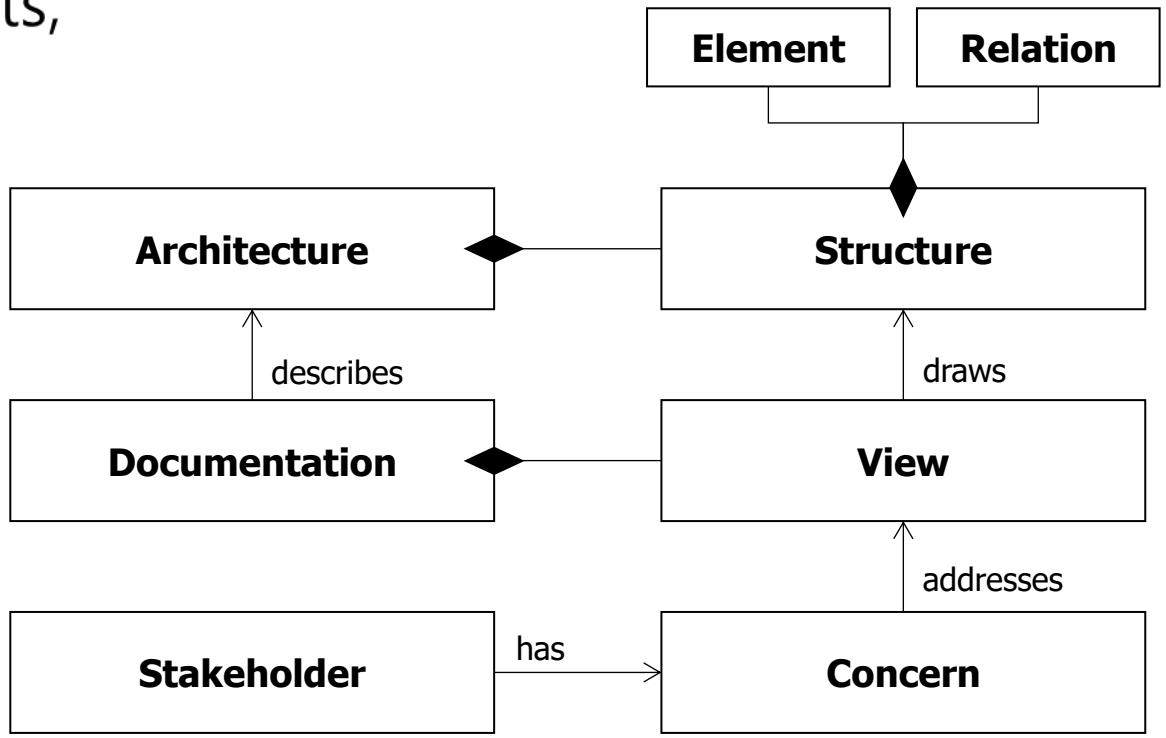
the set of structures

needed to reason about the system,

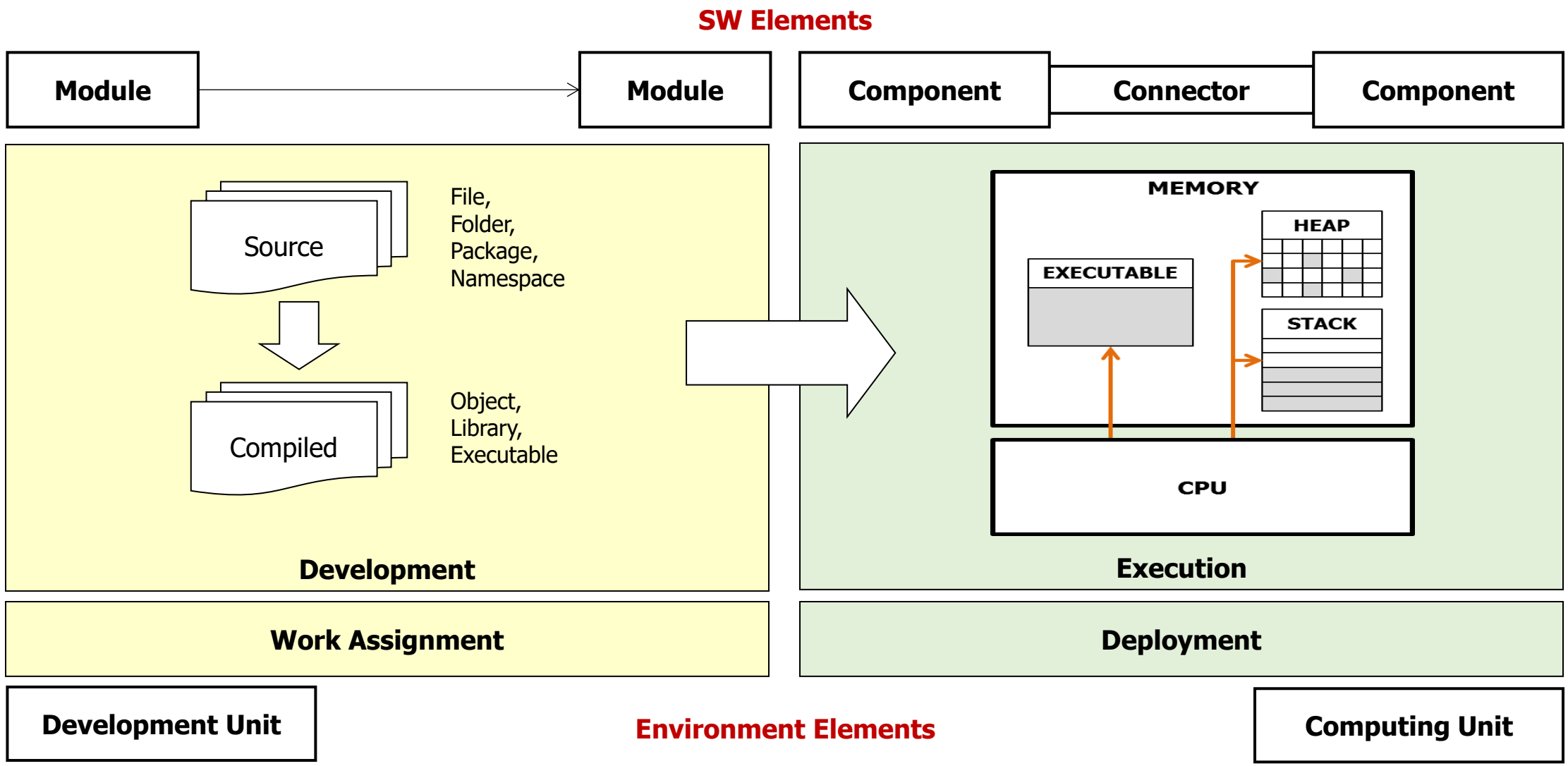
which comprise software elements,

relations among them,

and properties of both.



Kinds of Structures / Views



Module View

Elements	• **Modules**, which are implementation units of software.
Relations	• **Is part of**, which defines a part/whole relationship between the submodule the part-and the aggregate module the whole. • **Depends on**, which defines a dependency relationship between two modules. Specific module views elaborate what dependency is meant. • **Is a**, which defines a generalization/specialization relationship between a more specific module and a more general module.
Constraints	• Different module views may impose specific topological constraints, such as limitations on the visibility between modules.
Usage	• **Blueprint for construction of the code** • **Change-impact analysis** • Planning incremental development • Requirements traceability analysis • Communicating the functionality of a system and the structure of its code base • **Supporting the definition of work assignments, implementation schedules, and budget information** • Showing the structure of information that the system needs to manage

Layered Style

Overview	• The layered style puts together layers (groupings of modules that offer a cohesive set of services) in a **unidirectional allowed-to-use relation** with each other.
Elements	• **Layer**. The description of a layer should define what modules the layer contains.
Relations	• **Allowed to use**, which is a specialization of the generic **depends-on** relation. The design should define the layer usage rules (for example, "A layer is allowed to use any lower layer.") and any allowable exceptions.
Constraints	• Every piece of software is allocated to exactly one layer. • There are at least two layers (typically three or more). • The allowed-to-use relations should not be circular (that is, a lower layer cannot use a layer above).
Usage	• Promoting modifiability and portability • Managing complexity and facilitating the communication of the code structure to developers • Promoting reuse • Achieving separation of concerns

Layered Style - Example

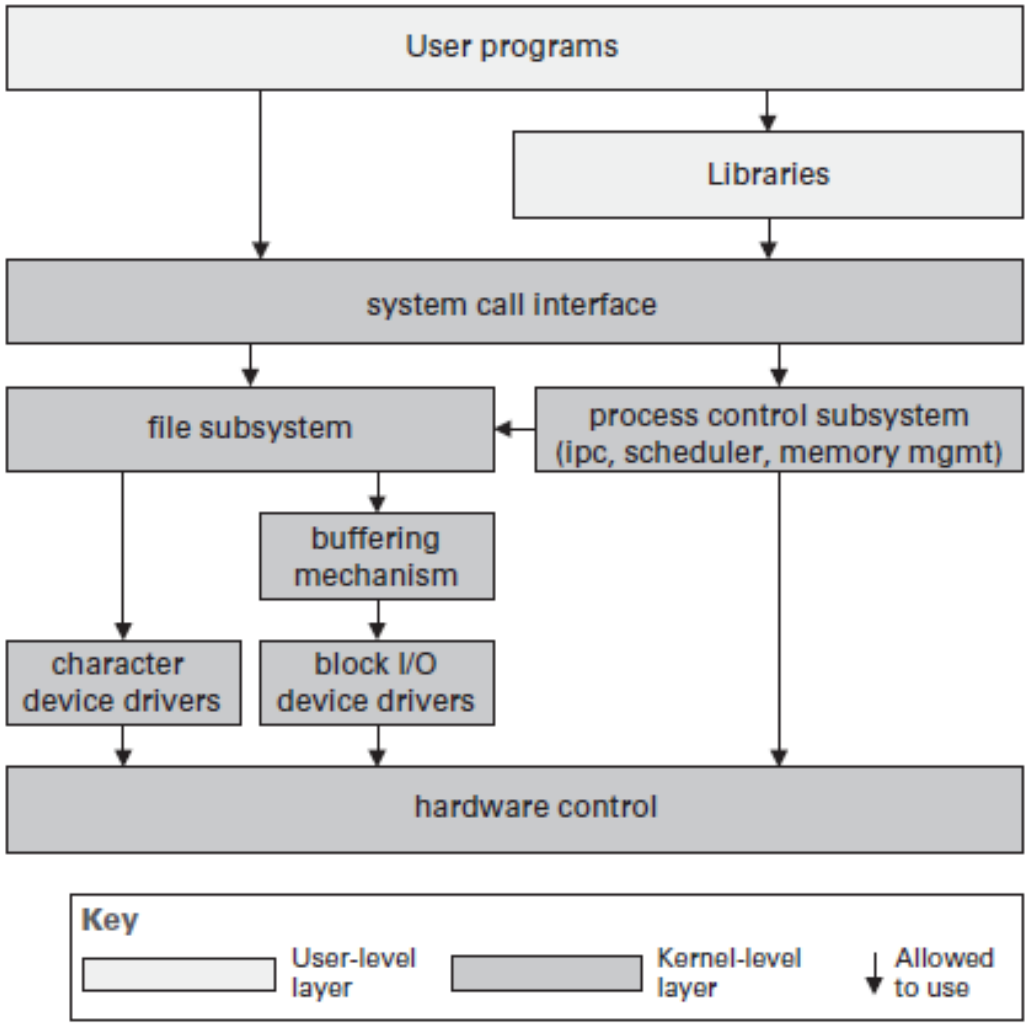


Figure 2.26
The primary presentation of a layered view of the UNIX System V operating system implementation (adapted from Bach 1986)

Work Assignment View

Overview	• The work assignment view describes the mapping of the software architecture to the teams in the development organization.
Elements	• **Software element**: a module. Properties include the required skill set and available capacity (effort, time) needed. • **Environmental element**: an organizational unit, such as a person, a team, a department, a subcontractor, and so on. Properties include the provided skill set and the capacity in terms of labor and calendar time available.
Relations	• **Allocated-to**. A software element is allocated to an organizational unit.
Constraints	• In general, the allocation is unrestricted; in practice, it is usually restricted so that one module is allocated to one organizational unit.
Usage	• For reasoning about distributed development and allocation of work to teams.

Work Assignment View - Example

ECS Element (Module)		
Segment	**Subsystem**	**Organizational Unit**
Science Data Processing Segment (SDPS)	Client	Science team
	Interoperability	Prime contractor team 1
	Ingest	Prime contractor team 2
	Data Management	Data team
	Data Processing	Data team
	Data Server	Data team
	Planning	Orbital vehicle team
Flight Operations Segment (FOS)	Planning and Scheduling	Orbital vehicle team
	Data Management	Database team
	User Interface	User interface team
. . .	. . .	. . .

Figure 5.6
Work assignment view using a tabular notation. The left two columns echo the system's module decomposition structure.

Component & Connector View

Elements	• **Components**. Principal processing units and data stores. A component has a set of ports through which it interacts with other components. • **Connectors**. Pathways of interaction between components. Connectors have a set of roles (interfaces) that indicate how components may use a connector in interactions.
Relations	• **Attachments**. Component ports are associated with connector roles to yield a graph of components and connectors. • **Interface delegation**. In some situations, component ports are associated with one or more ports in an "internal sub-architecture. The case is similar for the roles of a connector.
Constraints	• Components can only be attached to connectors, not directly to other components. • Connectors can only be attached to components, not directly to other connectors. • Attachments can only be made between compatible ports and roles. • Interface delegation can only be defined between two compatible ports (or two compatible roles). • Connectors cannot appear in isolation; a connector must be attached to a component.
Usage	• **Show how the system works.** • Guide development by specifying structure and behavior of runtime elements. • **Help reason about runtime system quality attributes, such as performance and availability.**

Client-Server Style

Elements	• **Client,** which is a component that invokes services of a server component. • **Server,** which is a component that provides services to client components. • **Request/reply connector,** which is used by a client to invoke services on a server. Request/reply connectors have two roles: a request role and a reply role. Connector properties may include whether the calls are local or remote, and whether data is encrypted.
Relations	• The **attachment relation** associates client service-request ports with the request role of the connector and server service-reply ports with the reply role of the connector.
Constraints	• Clients are connected to servers through request/reply connectors. • Server components can be clients to other servers. • Components may be arranged in tiers.
Usage	• Promoting modifiability and reuse by factoring out common services • Improving scalability and availability in case server replication is in place • Analyzing dependability, security, and throughput

Client-Server Style - Example

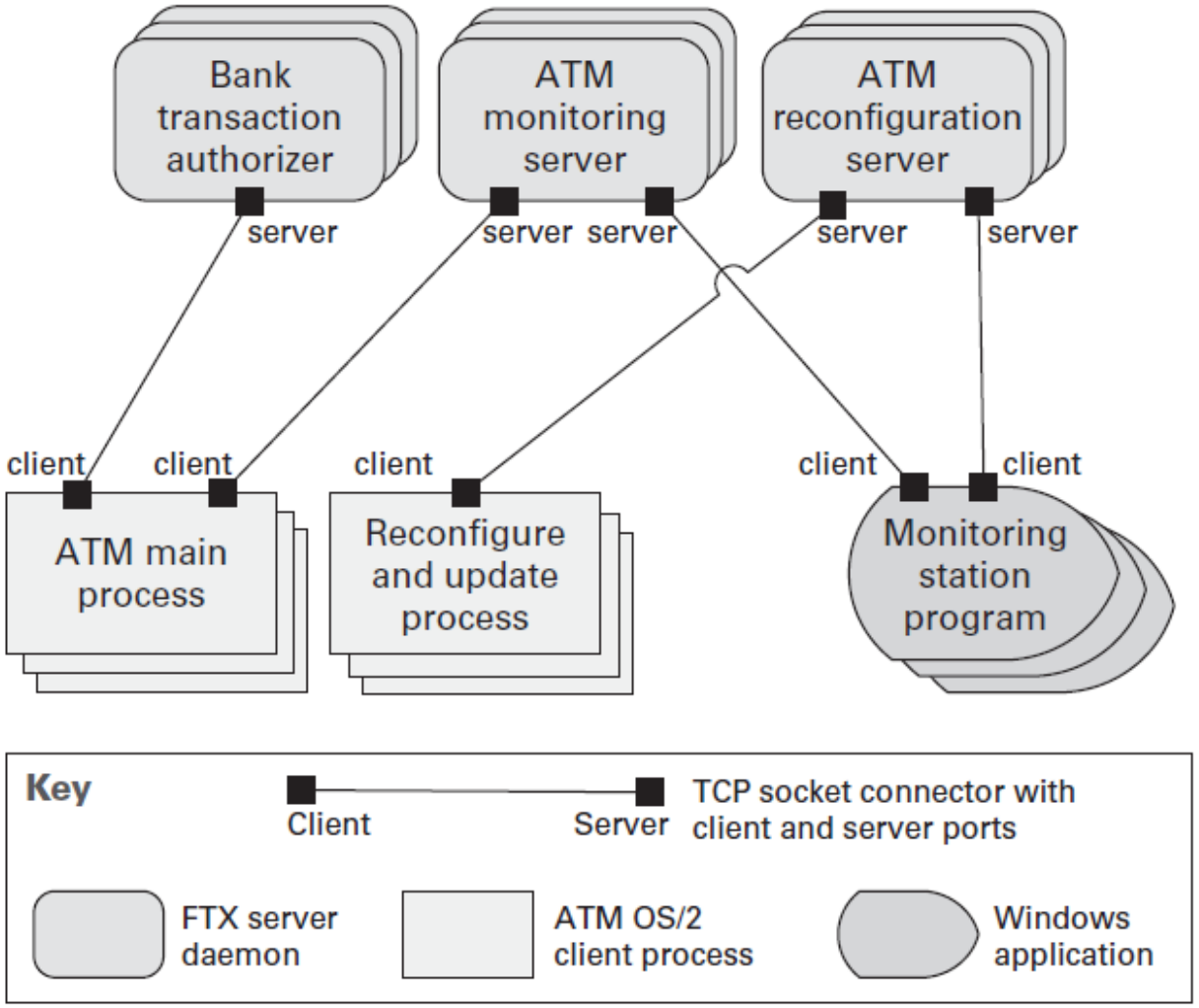
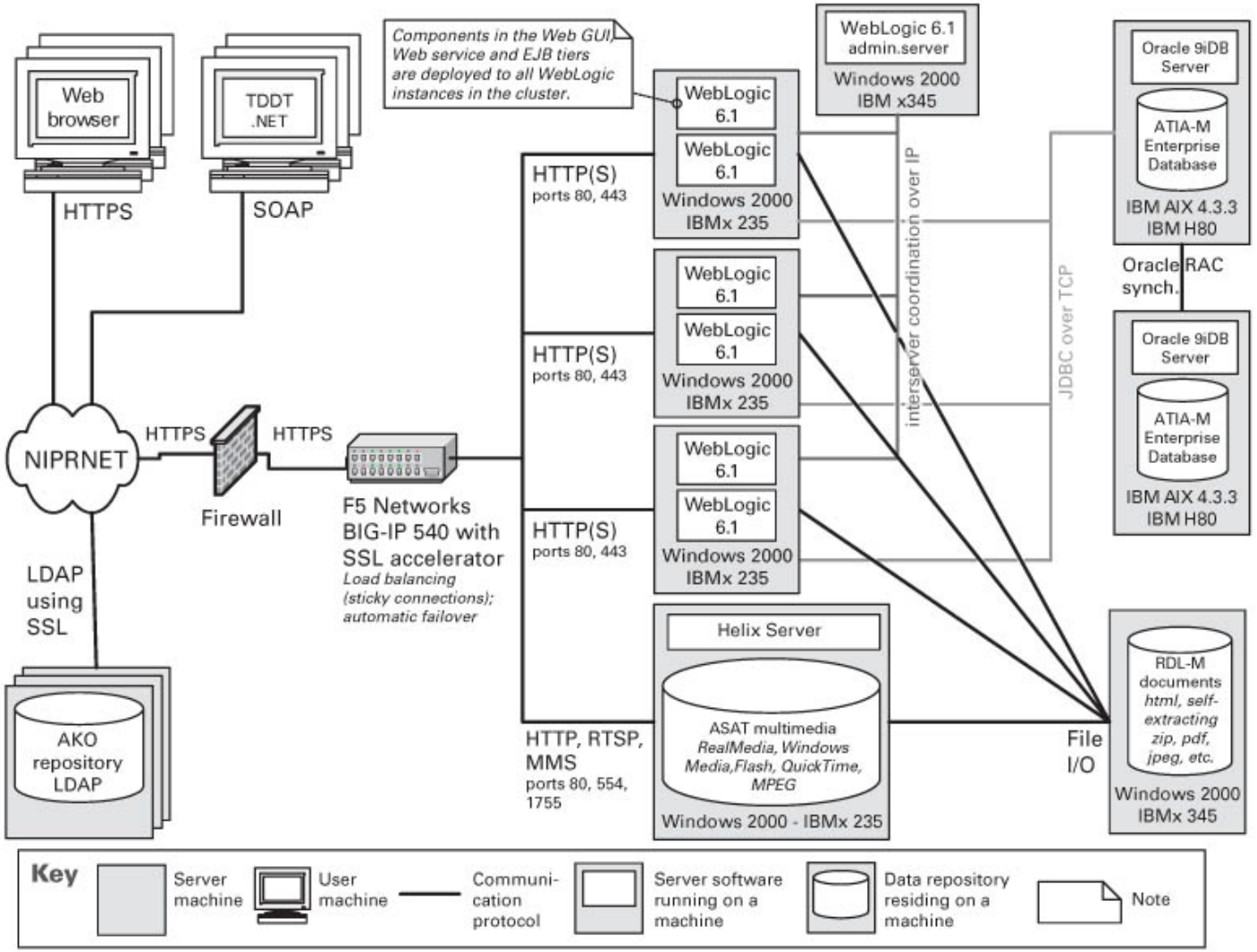


Figure 4.3
Client-server architecture of an ATM banking system. The `ATM main process` sends requests to `Bank transaction authorizer` corresponding to user operations (such as deposit, withdrawal). It also sends messages to `ATM monitoring server` informing the overall status of the ATM (devices, sensors, and supplies). The `Reconfigure and update process` component sends requests to `ATM reconfiguration server` to find out if a reconfiguration command was issued for that particular ATM. Reconfiguration of an ATM (for example, enabling or disabling a menu option) and data updates are issued by bank personnel using the `Monitoring station program`. `Monitoring station program` also sends peri-

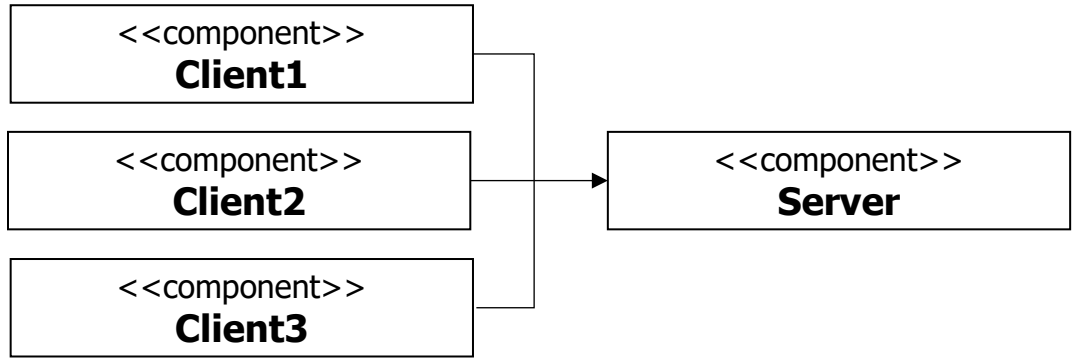
Deployment View

Overview	• The deployment view describes the mapping of components and connectors in the software architecture to the hardware of the computing platform.
Elements	• **Software element**: elements from a C&C view. Useful properties to document include the significant features required from hardware, such as processing, memory, capacity requirements, and fault tolerance. • **Environmental element**: hardware of the computing platform - processor, memory, disk, network(such as router, bandwidth, firewall, bridge), and so on. Useful properties of an environmental element are the significant hardware aspects that influence the allocation decision.
Relations	• **Allocated-to**. Physical units on which the software elements reside during execution. Properties include whether the allocation can change at execution time or not. • **Migrates-to, copy-migrates-to, and/or execution-migrates-to** if the allocation is dynamic. Properties include the trigger that causes the migration.
Constraints	• The allocation topology is unrestricted. However, the required properties of the software must be satisfied by the provided properties of the hardware.
Usage	• For reasoning about performance, availability, security and safety.

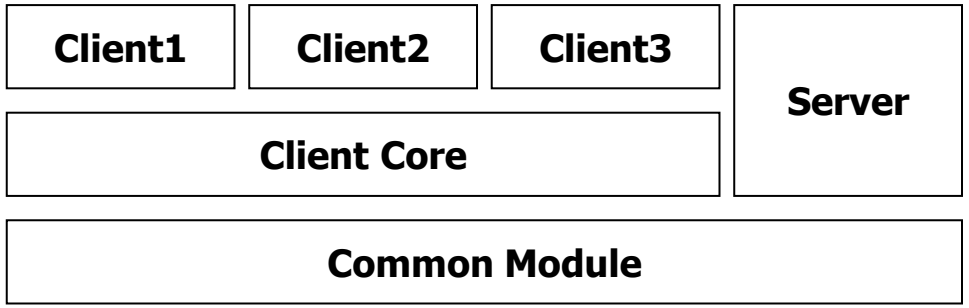
Deployment View - Example



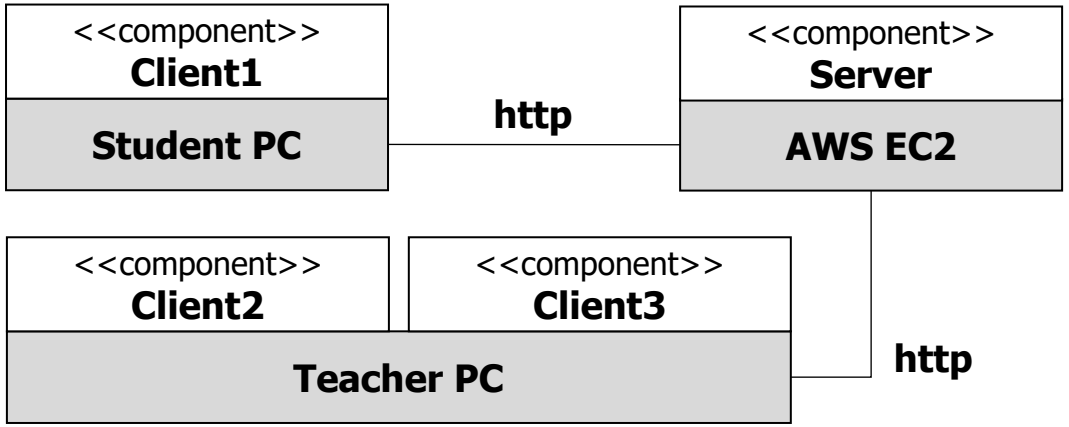
Kinds of Structures / Views



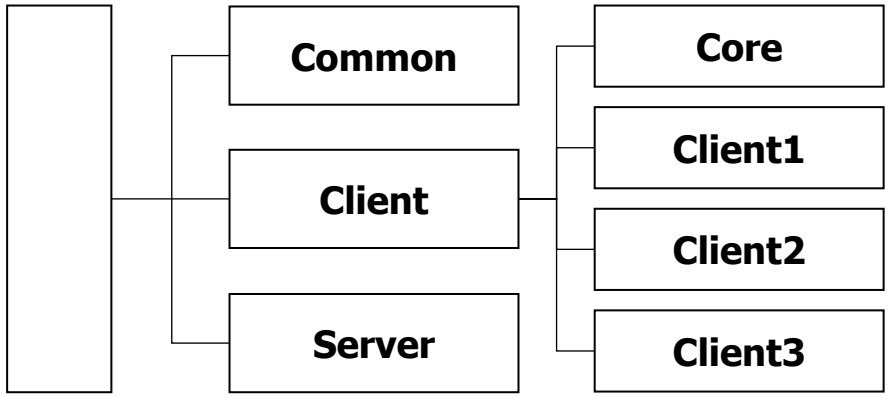
[Component & Connector View]



[Module View]

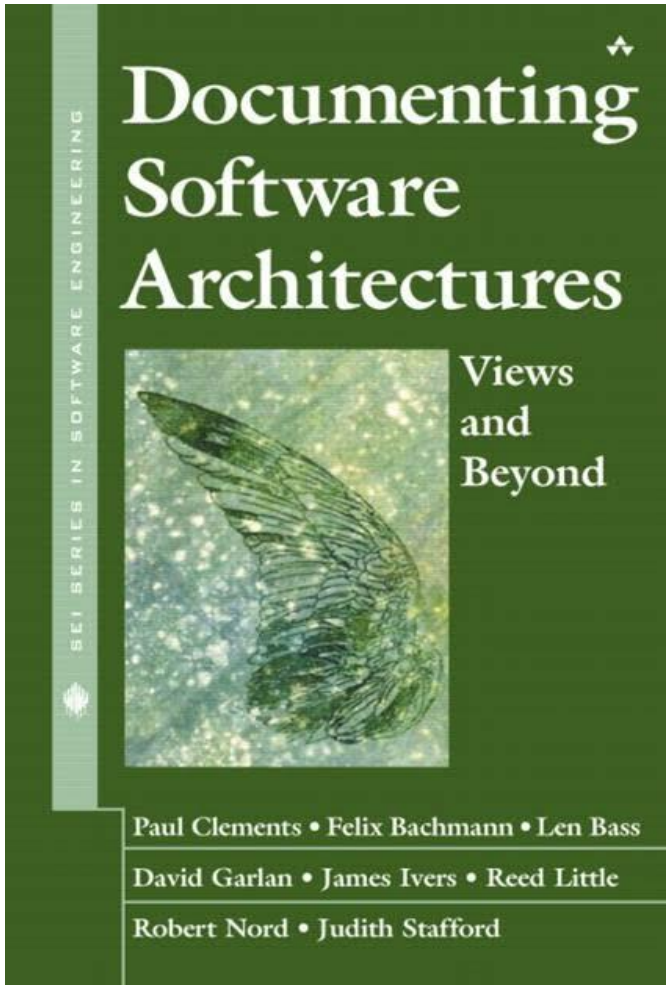


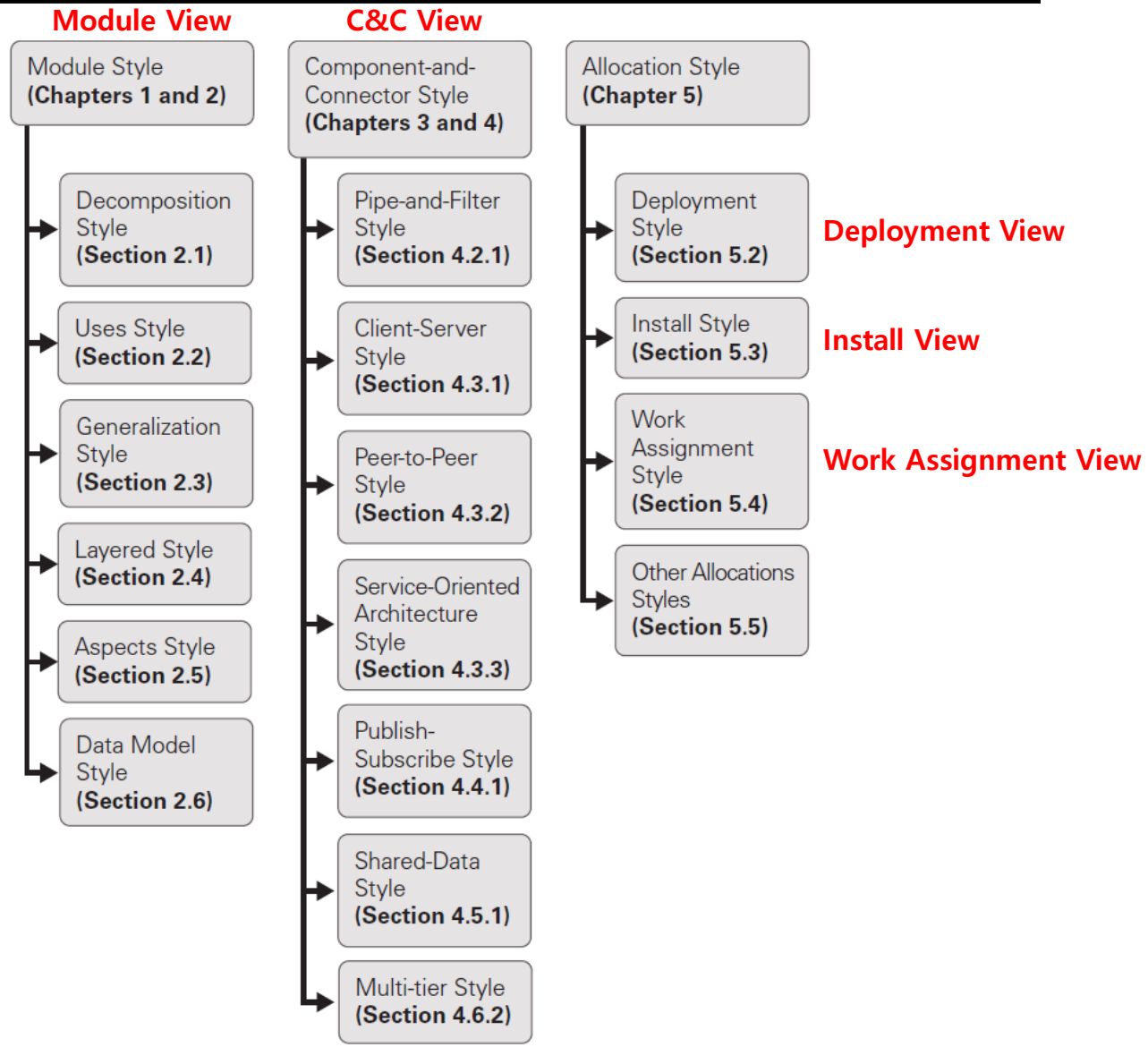
[Deployment View]



[Work Assignment View]

SW Architecture - Views and Beyond





실습1. 구조도

BY NC ND

Example 1.

repository: https://github.com/bosornd/arch-design-examples

- example-module-design

1) 실행/동작 측면의 구조도 그리기

2) 개발 측면의 구조도 그리기

Example 2.

repository: https://github.com/bosornd/arch-design-examples

- ros2_examples

1) 실행/동작 측면의 구조도 그리기

2) 개발 측면의 구조도 그리기

Example 3.

repository: https://github.com/bosornd/arch-design-examples

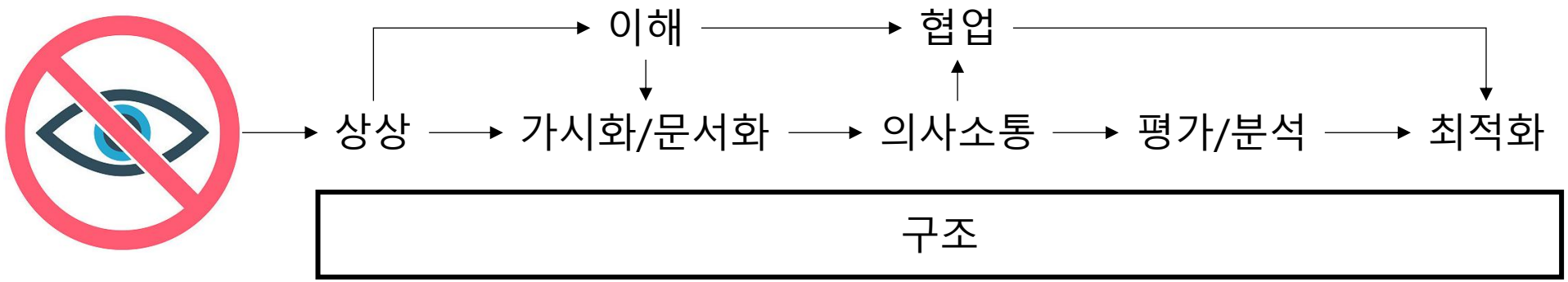
- k8s-example

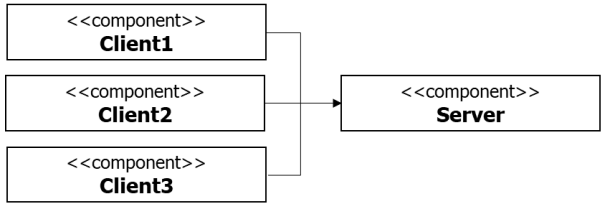
1) 실행/동작 측면의 구조도 그리기

2) 개발 측면의 구조도 그리기

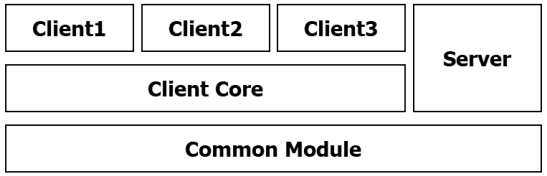
Role of Architect

- 구조도를 그린다.

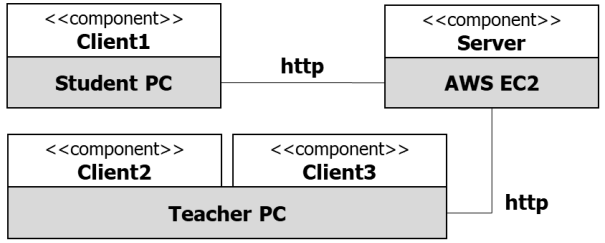




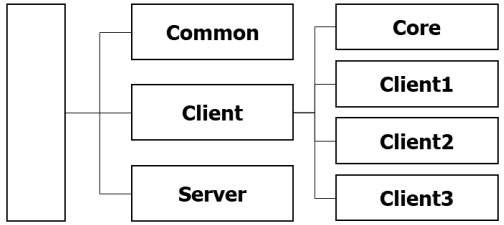
[Component & Connector View]



[Module View]



[Deployment View]



[Work Assignment View]

[Install View]

[Data View]

실습2. 재설계와 ADR

BY NC ND

Example 4.

repository: https://github.com/bosornd/arch-design-examples

- Lslidar_ROS2_driver

1) 실행/동작 측면의 구조도 그리기

2) 품질 시나리오 (성능) 작성하기

3) 대안 설계하기

4) ADR 작성하기

Example 5.

repository: https://github.com/bosornd/arch-design-examples

- Lslidar_ROS2_driver

1) 개발 측면의 구조도 그리기

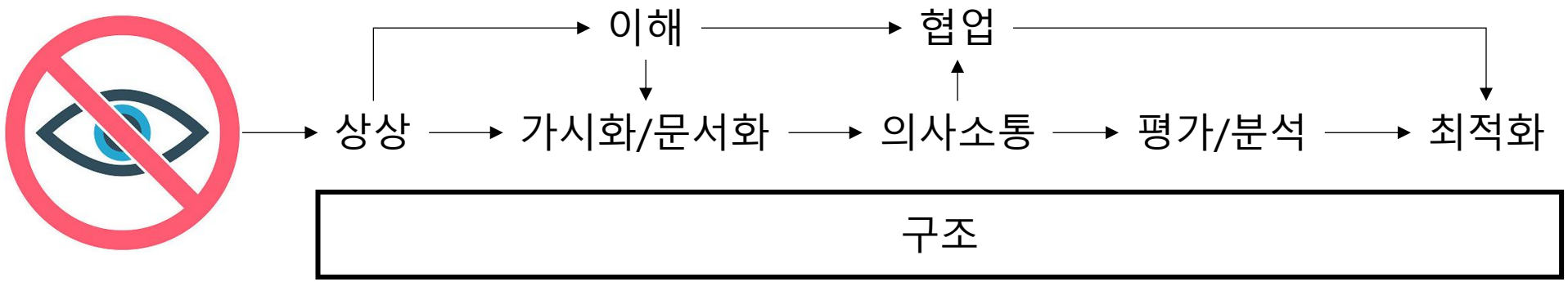
2) 품질 시나리오 (변경 용이성) 작성하기

3) 대안 설계하기

4) ADR 작성하기

Role of Architect

- 구조도를 그린다.

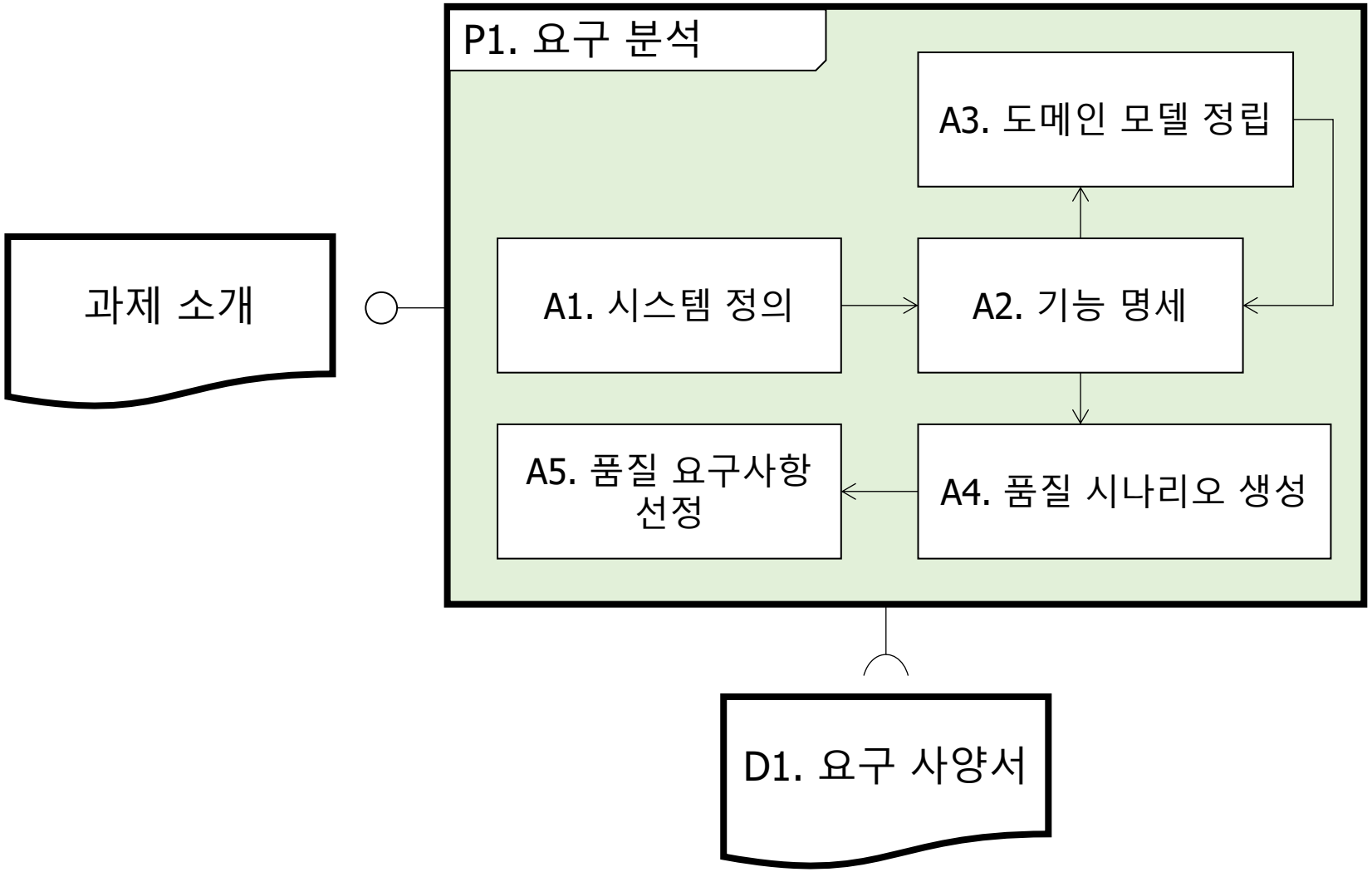


- 최적 설계/개발을 이끈다.
  - 구조 설계 활동

구조 설계 활동

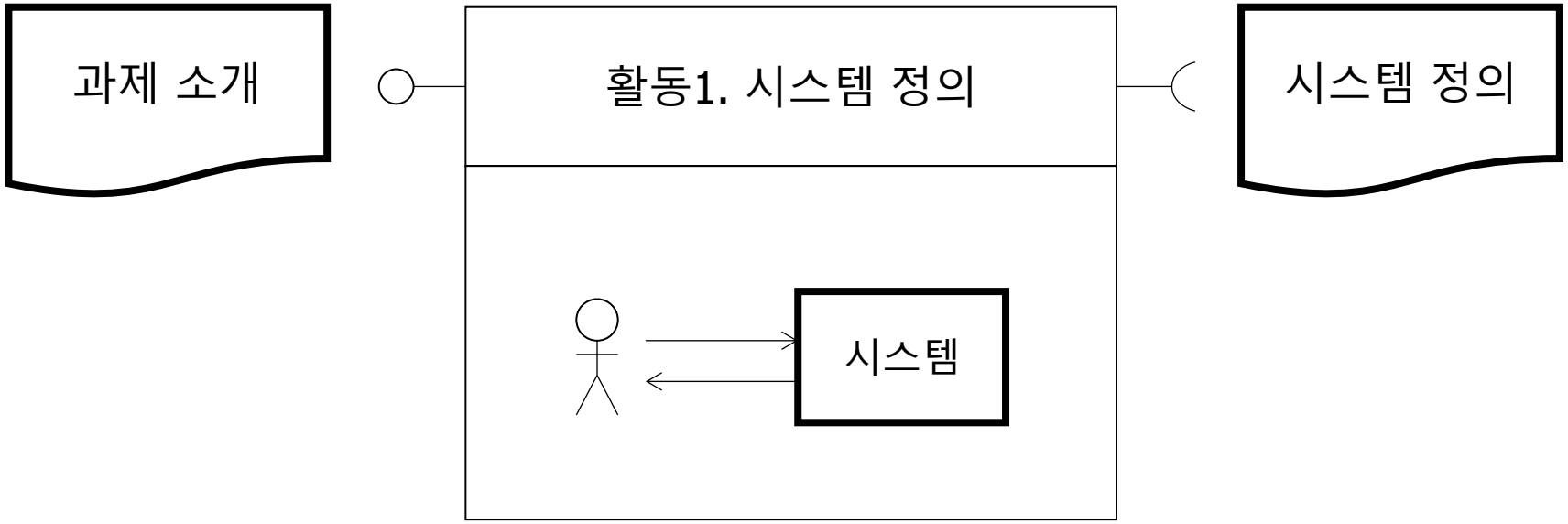
BY NC ND

P1. 요구 분석


P1. 요구 분석
A3. 도메인 모델 정립
과제 소개
A1. 시스템 정의
A2. 기능 명세
A5. 품질 요구사항 선정
A4. 품질 시나리오 생성
D1. 요구 사양서

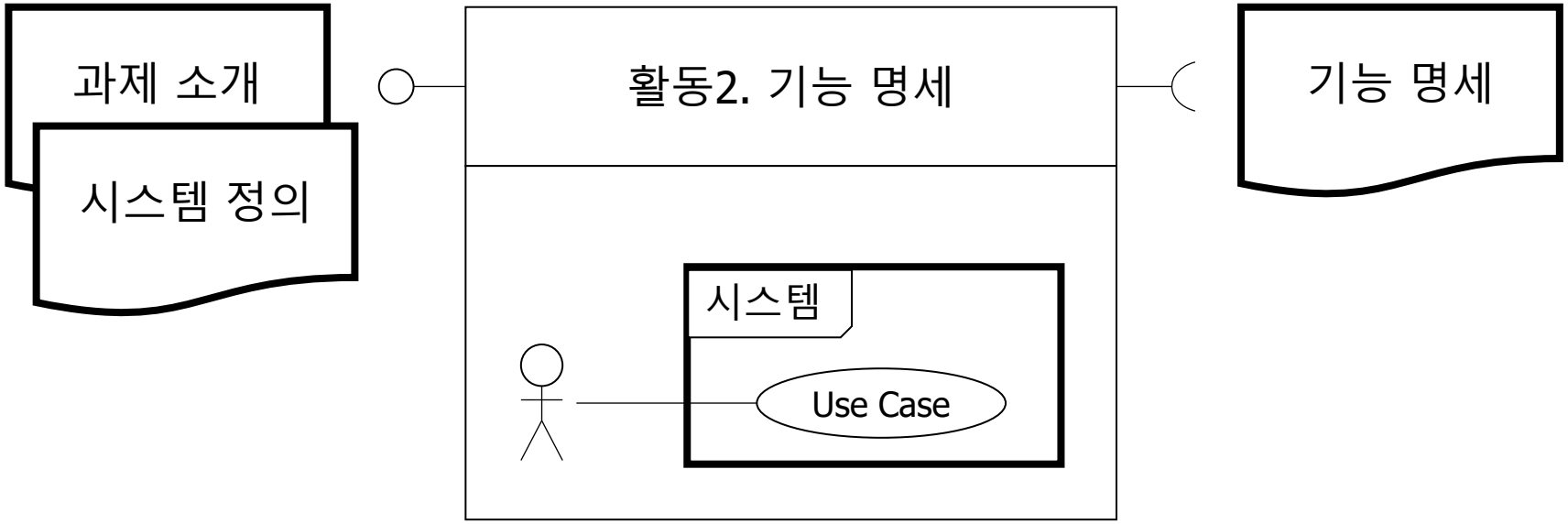
활동1. 시스템 정의

목적	개발하고자 하는 시스템의 경계를 정의한다. 시스템과 상호 작용하는 외부 시스템(이해관계자 포함)을 정의한다. 시스템과 외부 시스템과의 관계를 파악한다. 시스템의 동작 및 사업 환경을 파악한다.



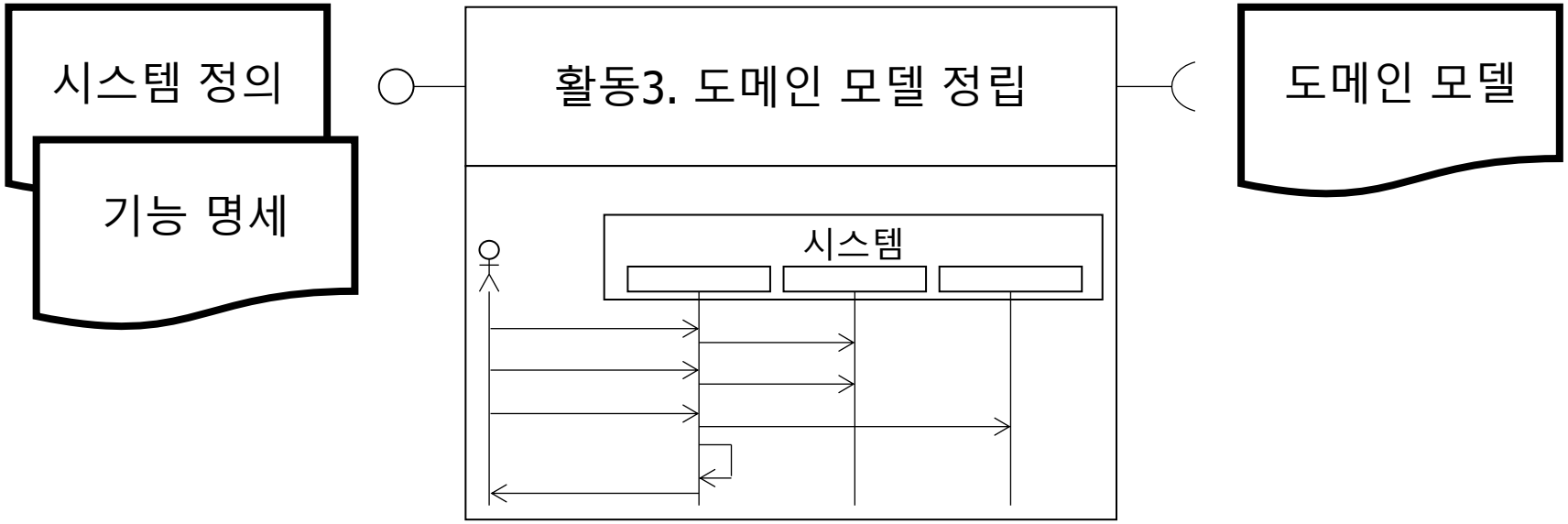
활동2. 기능 명세

목적	시스템의 기능을 명세한다. 외부 시스템의 요청에 대한 시스템의 반응을 명세한다.



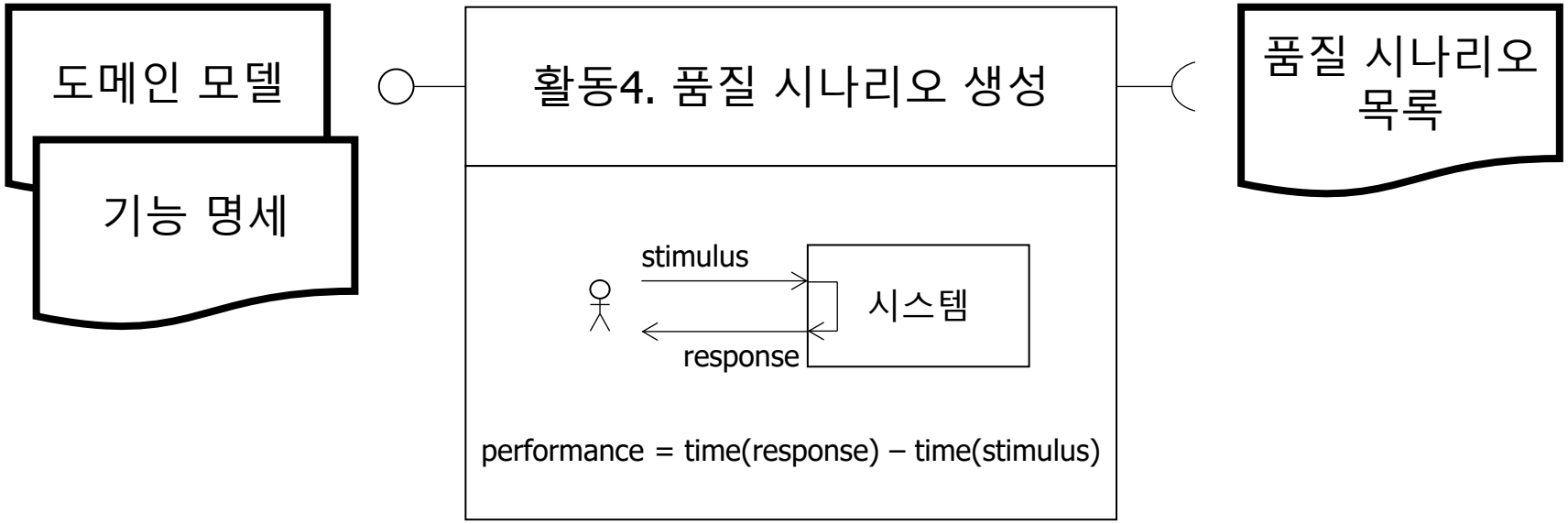
활동3. 도메인 모델 정립

목적	도메인 모델(개념적 구조)을 정립한다. 외부 시스템의 요청에 대한 시스템의 반응을 도메인 모델로 명세한다.



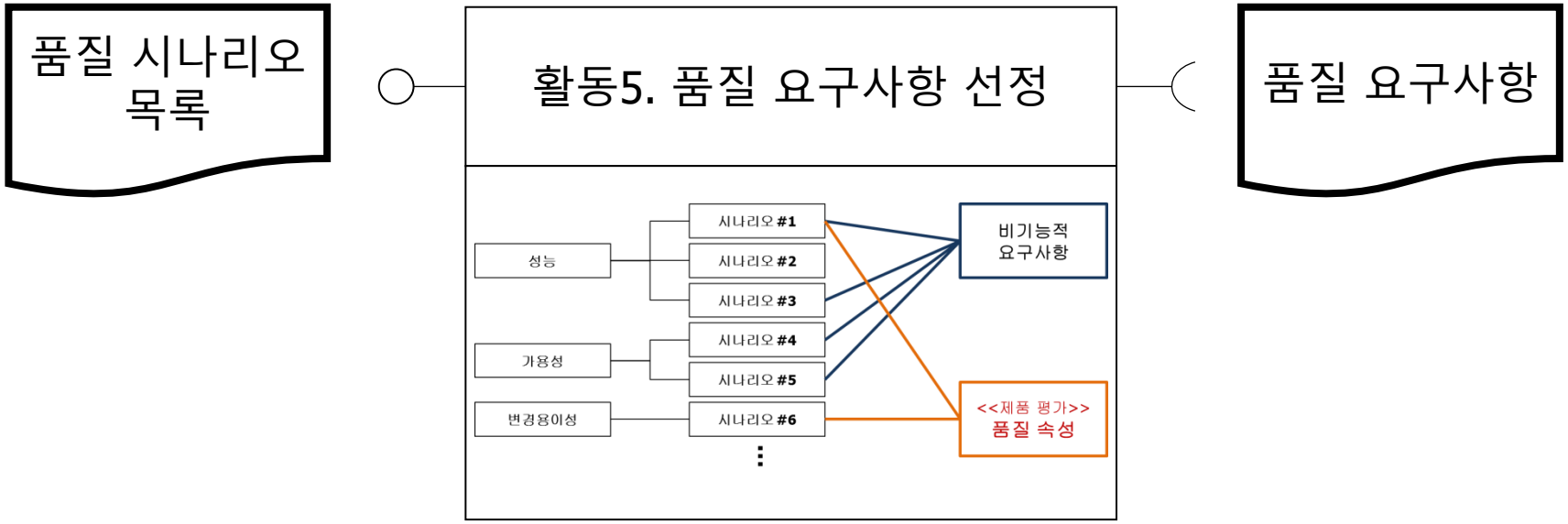
활동4. 품질 시나리오 생성

목적	시스템의 품질을 측정할 수 있는 시나리오를 생성한다.

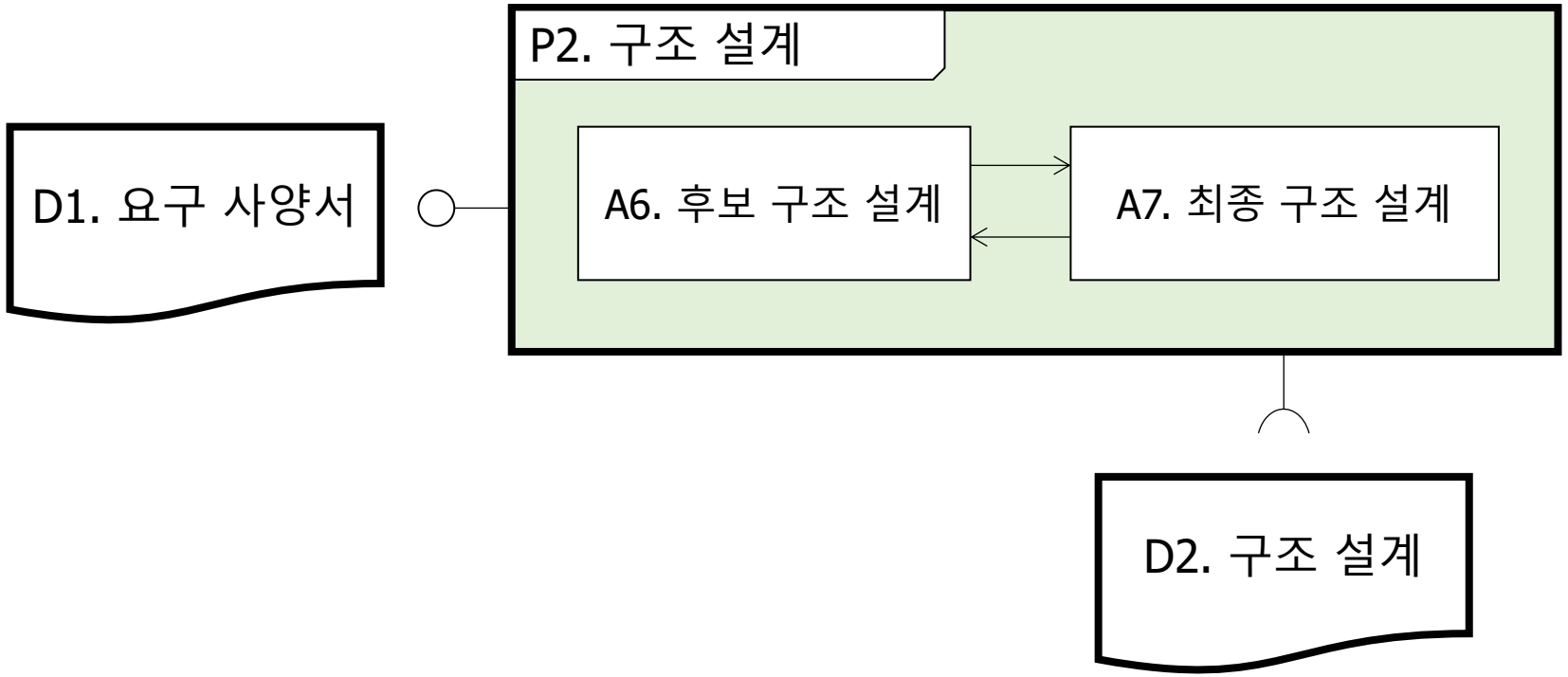


활동5. 품질 요구사항 선정

목적	품질 시나리오의 중요도 분석을 통해, 품질 요구사항(비기능적 요구사항과 품질 속성)을 선정한다.

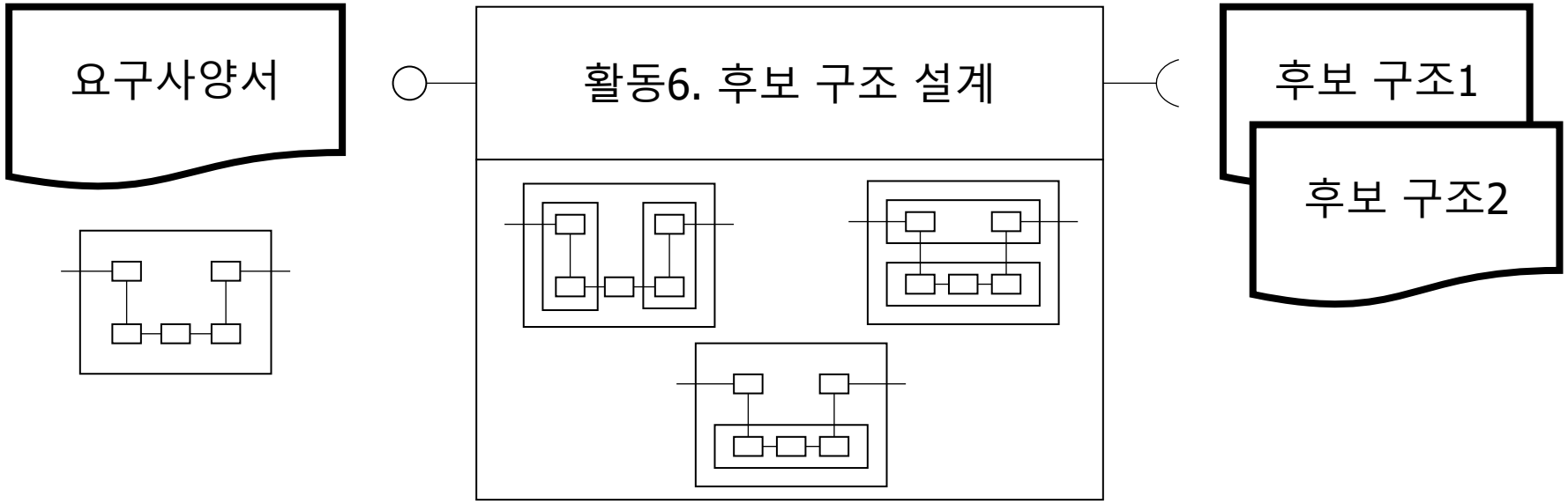


P2. 구조 설계(최적 설계)



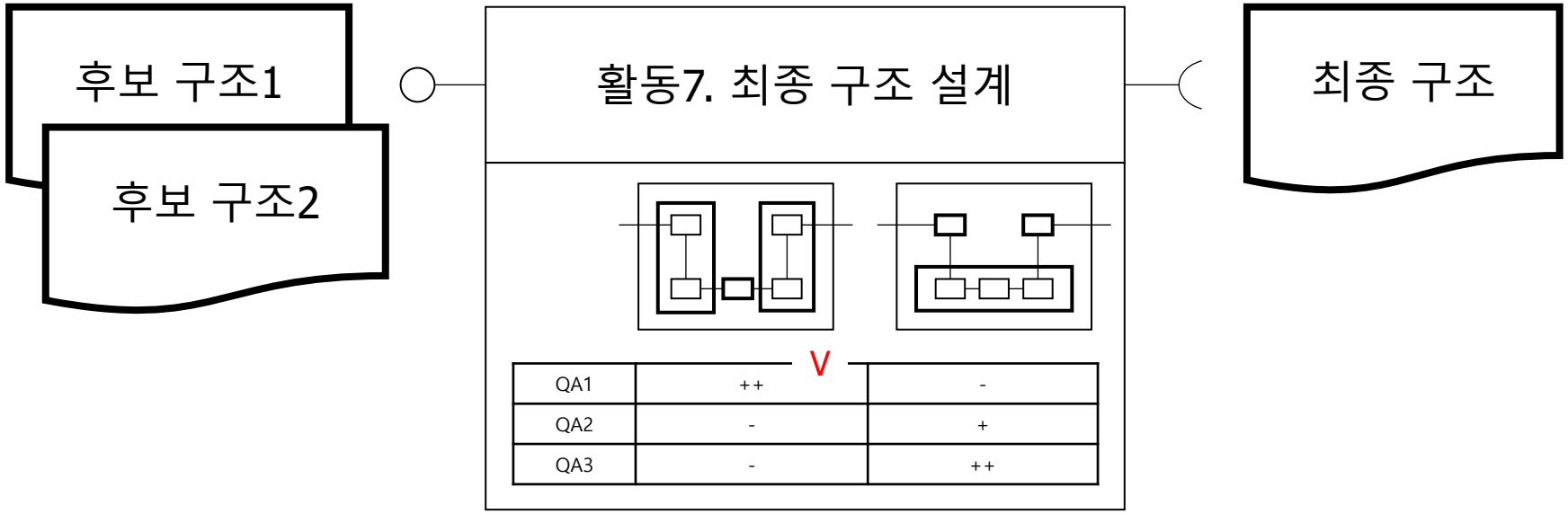
활동6. 후보 구조 설계

목적	도메인 모델(개념적 구조)을 실현하기 위한 시스템 구조를 설계한다. 품질 요구사항을 개선하는 후보 구조를 설계한다.

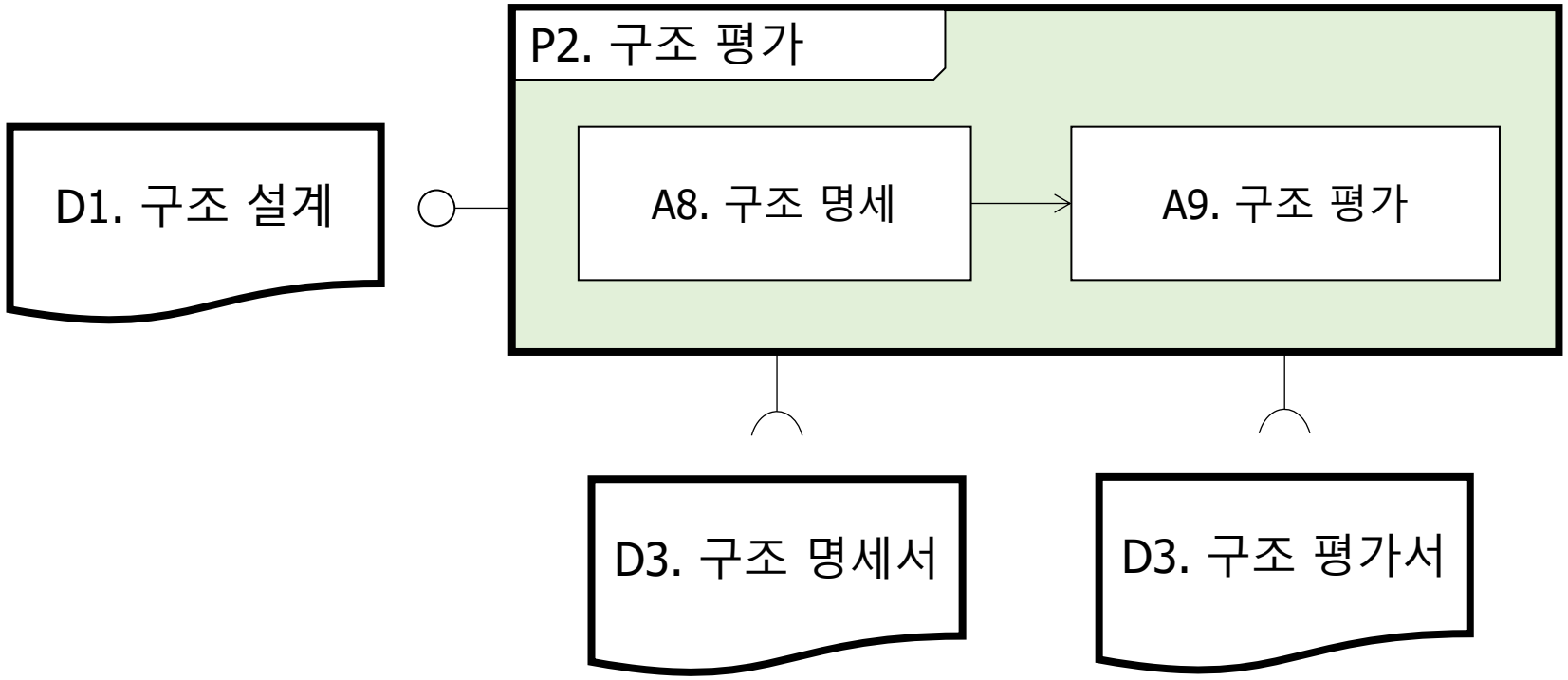


활동7. 최종 구조 설계

목적	과제의 요구사항에 가장 적합한 구조를 설계한다. 기능적/비기능적 요구사항을 만족하는 후보 구조 중에서, 품질 속성을 가장 잘 만족하는 후보 구조를 선정한다. 최종 구조의 단점을 보완한다.

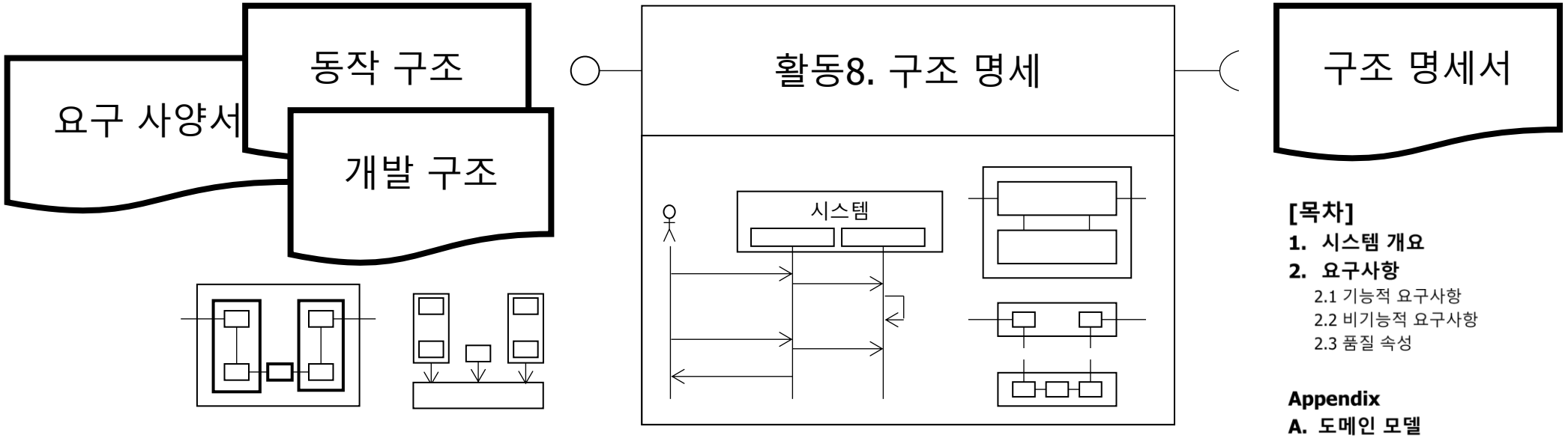


P3. 구조 평가



활동8. 구조 명세

목적	Stakeholder(특히 개발자)에게 설명하기 위한 구조 명세서를 작성한다. 시스템의 동작과 동작 특성(품질)에 대해서 설명한다. 상세 설계 및 구현을 위한 모듈 인터페이스를 명세한다. 시스템 개발을 위한 가이드를 제공한다.



[목차]

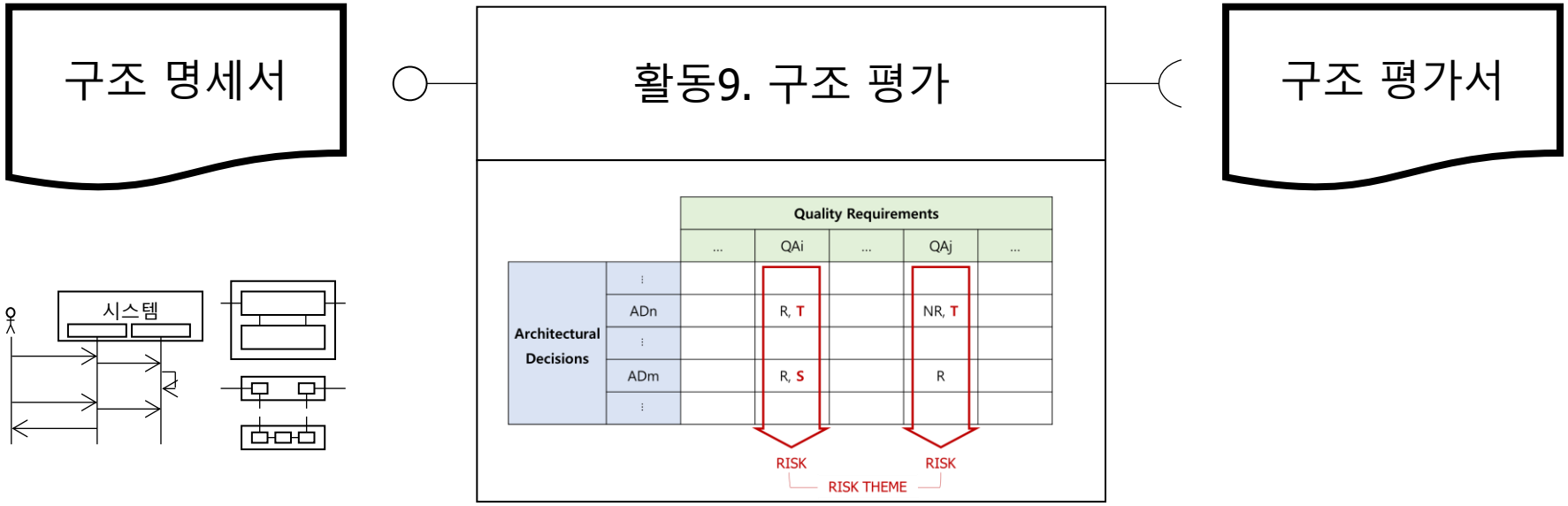
1. 시스템 개요
2. 요구사항
   2.1 기능적 요구사항
   2.2 비기능적 요구사항
   2.3 품질 속성
3. 시스템 구조
4. 모듈 사양

Appendix

A. 도메인 모델
B. 품질 시나리오
C. 품질 시나리오 분석
D. 후보 구조
E. 후보 구조 평가
F. 최종 구조 설계

활동9. 구조 평가

목적	시스템이 특정 목적에 부합하는 가를 확인한다. 위험을 분석하여 과제의 실패를 사전에 방지한다. 이해 관계자와 프로젝트 참여자 간의 커뮤니케이션을 향상시키고 공감대를 끌어낸다.



구조 설계 및 평가

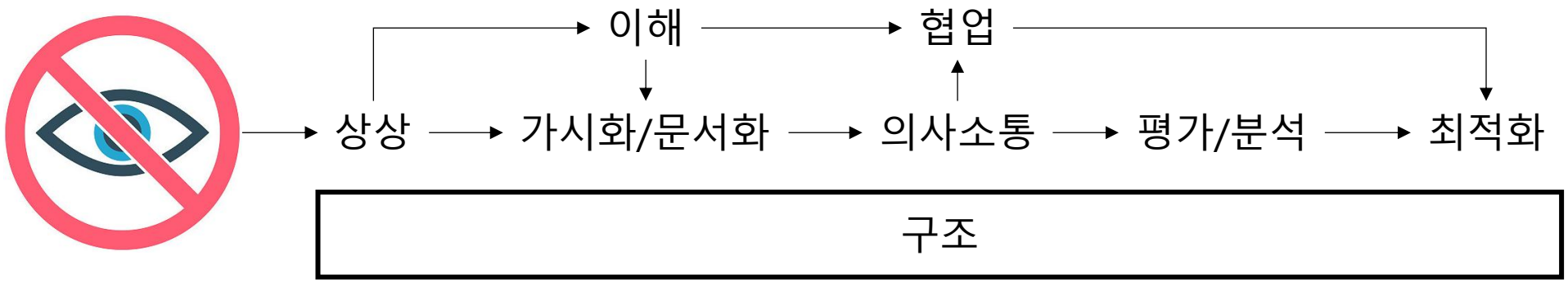
조 용 진

(drajin.cho@bosornd.com)

BY NC ND

Role of Architect

- 구조도를 그린다.



- 최적 설계/개발을 이끈다.
  - 품질 요구사항
  - 구조적 의사결정

Software Architecture

The software architecture of a system is

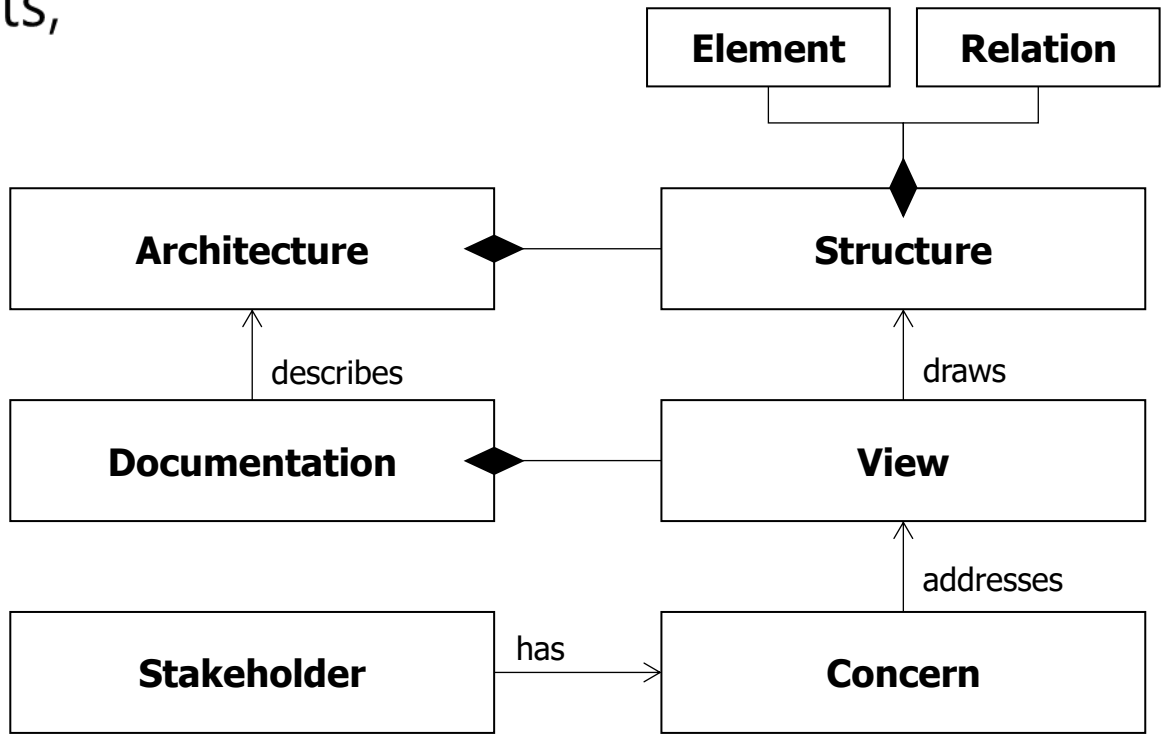
the set of structures

needed to reason about the system,

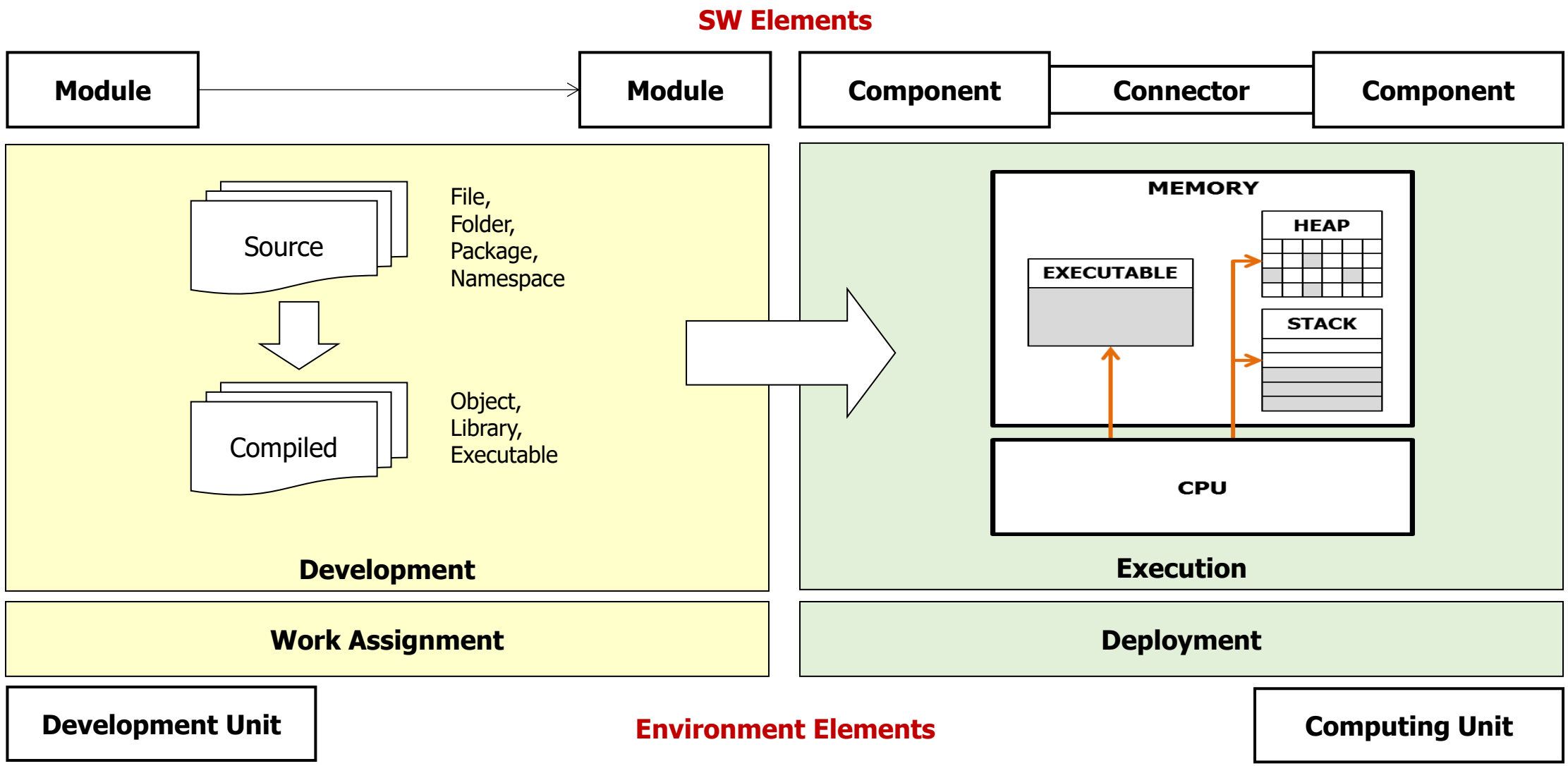
which comprise software elements,

relations among them,

and properties of both.



Software Architecture Views

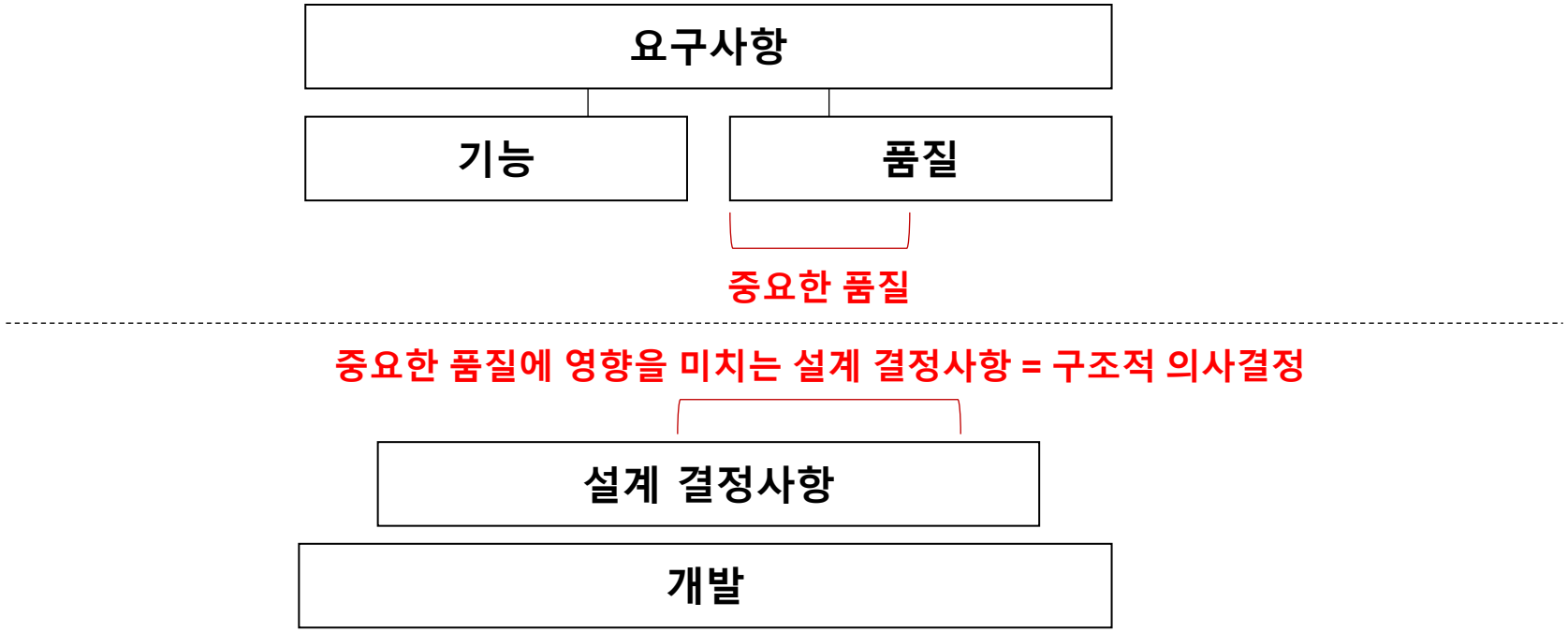


Architectural Decision

구조적 의사결정은 중요한 품질에 영향을 미치는 설계 결정사항이다.

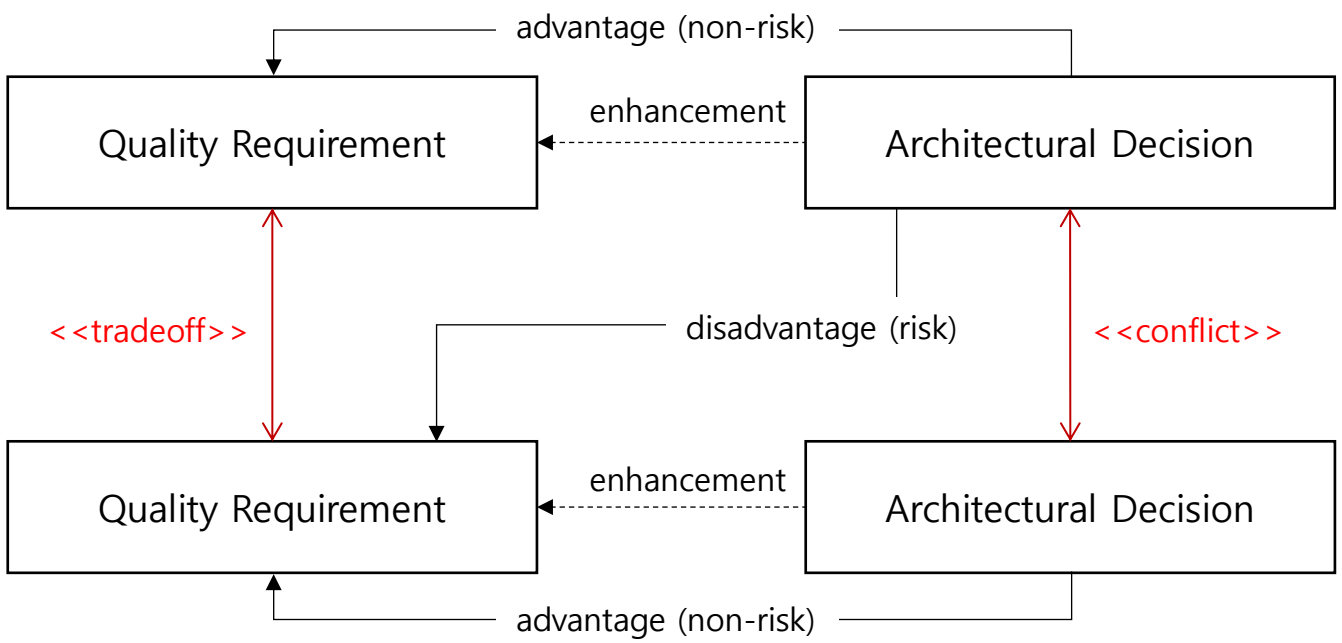
품질 요구사항과 구조적 의사결정의 관계를 관리하는 것이 필요하다.

- 품질 요구사항이 변한다. 최적화된 설계 결정사항의 채택이 필요하다.
- Tradeoff로 RISK가 있는 품질 요구사항에 대한 완화가 필요하다.



ADR (Architecture Decision Records)

It's better to manage relationships between quality requirements and architectural decisions (candidate architectures).



-- ADR--

Title

Context and Problem Statement

Decision Drivers

Decision Outcome

Considered Options

Pros and Cons of the Options

소프트웨어 아키텍트가 되는 좋은 습관

1. **최소 10% 시간 투자하기**
   10% 시간 낭비가 아닌 최소한의 투자
2. **팀원과 함께하기**
   다양한 관점과 집단 지성의 활용
3. **중요한 것에 집중하기**
   효율적이고 영향력 있는 아키텍트가 되는 길
4. **최적 추구하기**
   나중에 변경되지 않을 만큼 완전한 설계 추구
5. **작은 작업부터 연습하기**
   작은 작업은 완벽한 훈련장
6. **설계 공유하기**
   공유하고 리뷰하며 설계 연습을 확장
7. **열린 마음 유지하기**
   서로의 어려움을 인정하고 WIN-WIN을 위해 노력

https://www.udemy.com/course/habits-of-architect